

UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2024/2025



Wholly owned by UTAR Education Foundation
(Co. No. 578227-M)
DU012(A)

UCCD 1004 PROGRAMMING CONCEPTS AND PRACTICES

ASSIGNMENT 2

Group 51			
Name	ID	Programme	UTAR Email
Jazzwin Chang Tze Swen	2304267	CS	jezzwinchang@1utar.my
Leow Hui Yu	2304095	CS	l.huiyu19@1utar.my
Tan Yi Hui	2304970	CS	yihui.tan19@1utar.my

Task Division

	Name	Role	Modules
1.	Jazzwin Chang Tze Swen	Fix Errors	A function to read (ifstream) all the details available in "XX_paymentCheckIn.txt"
			A function to let users perform payment for flight tickets
			A function to allow users to check-in by providing the passenger(s) details
2.	Leow Hui Yu	Combine Code	A function to read (ifstream) all the user details exist in "user.txt"
			A function to register the users and include them into the records
			A function to validate both the username and password, ensuring secure user login access to the program.
3.	Tan Yi Hui	Error Checking	A function to read (ifstream) all the booking details of the passengers saved in "XX_Booking.txt"
			A function to perform booking of flight tickets and save into "XX_Booking.txt"
			A function to edit the booking details of the passengers

Objectives:

To develop and design a complete, user-friendly online booking system for JSJK Airline that allows users to conveniently:

1. Search and book flights options based on their travel needs
2. Make secure tickets reservations and payments
3. Perform online check-ins
4. Access and print detailed invoices after payment for their records.

Pseudocode

Define a structure named passengerStatus containing totalticketprice,paymentstatus,passengerdetails and checkinstatus.

Define a structure named checkIndetails containing passenger full name, passport number ,contact person full name and phone number.

Define bookingList that stores the passenger names, departure and return flight details.

Define passengerdetails as a variable of checkIndetails.

Define currentUserstatus as a variable of passengerStatus.

Define integer for number of tickets the current user has named noTickets, default value as 0.

Start Main

Loop until user input exit trigger

 Display register/login menu.

 Prompt for choice.

 If user choose to Register, then:

 Function call registration().

 Prompt user on confirmation to proceed to login.

 If user declines, then:

 End Main

 End if

 Else if user chooses to Login, then:

 Function call readUser to store the details of existing users into a list.

 Function call login.

 If login returns a number 1, then:

 Display message stating login successful.

 End Loop

 Else if login returns 2,then:

 Display message stating incorrect password.

 Else if login returns 3, then:

 Display message stating incorrect username.

 Else

 Display message stating incorrect username and password.

 End if

 Prompt user on confirmation to proceed to register/login menu.

 If user declines,then:

 End Main.

 End if

 Else if user chooses to Quit, then:

 End Main.

 Else

 Display error message stating invalid choice.

 End If

Define cont = 1

Repeat until cont is not 1

 //currentUser is the variable that stores the name of the program's current user

 Display currentUser

 Function call menu() to show options;

 Function call readBooking(bookingList, noTcikets) to store the booking details of currentUser.

Function call readPaymentCheckIn(currentUserstatus) to store the payment and check in details.

Prompt use to choose option.

If user chooses to make a booking, then:

Function call performBooking().

Else if user chooses to edit their booking, then:

Function call editBooking(bookingList, noTicktes).

Else if user chooses to make payment, then:

Function call payment().

Else if user chooses to check in, then:

Function call checkIn(currentUserstatus).

Else if user chooses to print invoice, then:

Function call printInvoice(bookingList, currentUserstatus).

Else if user chooses to quit.

Update cont = 0.

End if

If user chooses 1 or 2, then:

Open text file named "<currentUser>_paymentCheckin.txt" and

"<currentUser>_invoice.txt" for writing.

In payment check in file, Empty the passenger details and update the payment status as Not Paid and check in status as Not Checked In.

In invoice file, empty the content.

Close both files.

End if

Check if cont is 1.

End Main

#Sub-Functions:

Function registration():

Start

Initialize a new user

Initialize an array to store existing users

Set userCount to 0

Read existing users from file into users array

Set to assume no users exists before in the record

Display key in registration details message

Star a loop and execute at least once until valid:

Print to let user enter for first name

Read the user input for first name

If input is empty:

Show error message

Else if invalid format (non-letter characters):

Show error message

Stop the loop while it is not empty and valid

Start a loop and execute at least once until valid

Print to let user enter for last name

Read the user input for last name

If input is empty:

Show error message

Else if invalid format (non-letter characters):

```

        Show error message
    Stop the loop while it is not empty and valid

    Start a loop and execute at least once until valid
        Print to let user enter for mobile number
        Read the user input for mobile number
        If input is empty:
            Show error message
        Else if not numeric or not at least 10+ digits:
            Show error message
    Stop the loop while it is not empty and valid

    Start a loop and execute at least once until valid
        Print to let user enter for email
        Read the user input for email
        If input is empty:
            Show error message
        Else if invalid format:
            Show error message
    Stop the loop while it is not empty and valid

    Start a loop and execute at least once until valid
        Print to let user enter for username
        Read the user input for username
        If input is empty:
            Show error message

        Start a loop to loop through all the user's username character in the records
            If the same username exists before in the records file
                Break the loop
            If username already exists in users:
                Display username already exists message
        Stop the loop while it has gone through all the username with the user count
        Else if invalid format:
            Show error message
    Stop the loop while it is not empty and valid

    Start a loop and execute at least once until valid
        Print to let user enter for password
        Read the user input for password
        If input is empty:
            Show error message
        Else if does not meet criteria (min 8 chars, 1 symbol, 1 uppercase, 1 digit):
            Show error message
    Stop the loop while it is not empty and valid

    Repeat until valid input:
        Print to ask user to confirm (y/n)
        Read the user input for confirmation choice
        If invalid input:
            Repeat to ask
        Else if:
            Stop the loop
    If user chooses 'n':
        Display "Registration cancelled" message

```

Exit function

Open user file in append mode
Write user data line by line to the file
Close file
Display "Registration successful" message

End Function

Function read user ():

Start

Open user.txt for reading
Set user count to 0
Repeat while not at the end of file and count < SIZE
 Read line by line from the file
 First Name
 Last Name
 Mobile Number
 Email
 Username
 Password
 If unable to access and open the file:
 Exit the function
 Store user record in usercount and increase the count
Close the file

End Function

Function login()

Start

Set assumes no initial username found
Set assumes no initial password found
Print to let user enter for username
Read the input username from user
Print to let user enter for password
Read the input password from user

If input is empty:
 Show error message
 Return both incorrect

Start a loop to loop through all the user's username & password character in the record
 If username and password match:
 Return both correct
 Else if only username correct:
 Set username found to true
 Else if only password correct:
 Set password found to true
Stop the loop while it has gone through all the username and password with the user count

If username found:
 Return username correct, password incorrect
Else if password found:
 Return password correct, username incorrect
Else:
 Return both incorrect

End function

Define a function menu()

 Display menu

Define a function FlightSchedule()

 Display flight schedule

Define a function performBooking() for user to input the passenger details and save it in the file
Start

 Call struct array bookingDetails bookList[SIZE]

 Open text file named "<currentUser>_Booking.txt" for reading

 If the file is opened, then:

 Display the booking record had detected, choose to edit or not

 Prompt the user to enter the selection (y)es or (n)o

 If the selection is (y)es

 Call function editBooking(bookingList, noTickets)

 else

 Return to main function

 Close the file

 Create text file named "<currentUser>_Booking.txt" for saving.

 Call function FlightSchedule()

 Prompt the user enter the number of tickets which they like to book

 Loop through all the input of the passenger information from index = 0 to index = number of
tickets:

 Display the current number of passenger

 Prompt the user enter the first name, last name

 Save the first name into the file

 Save the last name into the file

 Repeat until the user input the details is same with the first passenger

 Prompt user to enter the departure flight num

 Save the departure flight number into the file

 Repeat until the user input the details is same with the first passenger

 Prompt user to enter the departure date

 Save the departure date into the file

 Repeat until the user input the details is same with the first passenger

 Prompt user to enter the departure time

 Save the departure time into the file

 Repeat until the user input the details is same with the first passenger

 Prompt user to enter the return flight num

 Save the return flight number into the file

 Repeat until the user input the details is same with the first passenger

 Prompt user to enter the return date

 Save the return date into the file

 Repeat until the user input the details is same with the first passenger

 Prompt user to enter the return time

 Save the return time into the file

 Close the file

 Display the message that your booking is successful

 Return a main function

Define a function readBooking(bookingDetails bookingList[], int& noTickets), pass the bookingList and the reference parameter of the number of tickets

Start

 Declare number of tickets equal to zero

 Open text file named "<<currentUser>>_Booking.txt" for reading

 If the file is opened

 While not at the end of file:

 Read out the first name and save into the struct array

 Read out the last name and save into the struct array

 Read out the departure flight number and save into the struct array

 Read out the departure date and save into the struct array

 Read out the departure time and save into the struct array

 Read out the return flight number and save into the struct array

 Read out the return date and save into the struct array

 Read out the return time and save into the struct array

 If the read file is failed

 Break the file

 Number of ticket + 1

 Close the file

 Else

 Return to the main function

Return to main function

Declare a function editBooking(bookingDetails bookingList[], int notickets), pass the bookingDetails and the number of tickets

Start

 Create text file named "<<currentUser>>_Booking.txt" for reading

 If the file cannot read

 Display the message no booking available

 Return to the main function

 Close the file

 Call function FlightSchedule()

 Prompt the user to enter the choice to change the name

 Display message to ask the name that user want to change

 Prompt the user to enter the original first name, last name

 Loop through all the name in the struct array from index = 0 to index = number of tickets

 If the original first name is found in the first name of the struct array and the original last name is found in the last name of the struct array

 Change the boolean founr the file into true statement

 Display the message name is found

 Prompt the user to enter the new first name, last name

 Prompt the user to enter the choice of the confirm amendment

 If the user confirm to amendment

 Save the new first name into the first name of the struct array

 Save the new last name into the last name of the struct array

 Break the loop

 If the name not found

 Display the message name is not found

 Else if the choice to change the name is (n)o

 Display the choice of the item to amend

 1. Departure Flight

 2. Departure Date

 3. Departure Slot

 4. Return Flight


```

5. Return Date
6. Return Slot
Prompt the use to enter the selection of the item to amend
Display the message follow the details of the first passenger
If the selection of the item to amend is 1
    Repeat until the user confirm the amendment
        Prompt the user to enter the new departure flight number
        Prompt the user to enter the choice of confirm amendment
        If the amendment is confirmed
            Save the new departure flight number into the depature flight
number struct array
    If the selection of the item to amend is 2
        Repeat until the user confirm the amendment
            Prompt the user to enter the new departure date
            Prompt the user to enter the choice of confirm amendment
            If the amendment is confirmed
                Save the new departure date into the depature date struct
array
    If the selection of the item to amend is 3
        Repeat until the user confirm the amendment
            Prompt the user to enter the new departure slot
            Prompt the user to enter the choice of confirm amendment
            If the amendment is confirmed
                Save the new departure date into the depature slot struct
array
    If the selection of the item to amend is 4
        Repeat until the user confirm the amendment
            Prompt the user to enter the new return flight number
            Prompt the user to enter the choice of confirm amendment
            If the amendment is confirmed
                Save the new return flight number into the depature flight
number struct array
    If the selection of the item to amend is 5
        Repeat until the user confirm the amendment
            Prompt the user to enter the new return date
            Prompt the user to enter the choice of confirm amendment
            If the amendment is confirmed
                Save the new return date into the depature date struct array
    If the selection of the item to amend is 6
        Repeat until the user confirm the amendment
            Prompt the user to enter the new return slot
            Prompt the user to enter the choice of confirm amendment
            If the amendment is confirmed
                Save the new return date into the depature slot struct array
Display the message your booking is successfully edited
Loop until all struct array from the index = 0 to index = number of tickets:
    Update the first name of the struct into the file
    Update the last name of the struct into the file
    Update the departure flight number of the struct into the file
    Update the departure date of the struct into the file
    Update the departure slot of the struct into the file
    Update the return flight number of the struct into the file
    Update the return date of the struct into the file
    Update the return slot of the struct into the file
Close the file

```

Return to main function
Define function readPaymentCheckIn(currentUserstatus), pass currentUserstatus by reference for manipulation in function.
Start

Open text file named "<currentUser>_paymentCheckin.txt" for reading.

If the file is opened, then:

While able to Read and store totalticketprice into currentUserstatus,do:

Discard any unwanted characters in the input buffer.

Read and store paymentstatus into currentUserstatus.

Loop through index=0 to index=noTickets-1:

Read and store passenger i's first and lastname,passportnumber,
contactperson's first and last name and mobile phone number into
currentUserstatus.

End Loop

Read in and store checkinstatus into currentUserstatus.

End while

End if

Close the paymentcheckin text file.

End function

Define function payment().

Start

//check if user has a booking due for payment.

Open text file named "<currentUser>_booking.txt" for reading.

If the file is not opened, then:

Display error message stating can't make payment when no booking is made.

End function.

Else if current user's payment status = Paid, then:

Display error message stating payment already made.

End function.

End if

//checking complete, user has a booking due payment.

Close the booking text file.

Display total flight tickets, departure and return flight's date, time and route and total amount due.

Prompt user for confirmation on continue for payment.

If confirmation = no, then:

End function.

End if

Display payment methods("1. Credit/ Debit card 2. Bank Transfer").

Prompt for choice on payment method.

Prompt user to enter total transfer amount.

If method choice = 1,then:

Prompt user to enter card holder's name, card number, card valid date and card
security code.

Else, do:

```

    Prompt user to enter bank choice, bank account number and pin number.
End if

Prompt user to enter transaction confirmation.

If confirmation = no, then:
    End function.
Else, do:
    Update currentUserstatus's paymentstatus as "PAID".

    Open text file named "<currentUser>_paymentCheckin.txt" for reading.
    While able to read a line from file into lines[count], do:
        If lines[count] = "Not Paid", then:
            Replace line[count] = "Paid".
            Increment count by 1.
        End if
    End While
    Close the paymentcheckin text file.

    Open text file named "<currentUser>_paymentCheckin.txt" for writing.
    Loop through index = 0 to index = count, do:
        Write elements with corresponding index in lines list into text file.
    End loop
    Close the paymentcheckin text file.

End if
End function.

Define function checkIn(currentUserstatus), pass currentUserstatus by reference for manipulation in
function.
Start
    //User cannot check in 1. if no booking is made or 2. the booking is unpaid r 3. payment is
    made and check in is done.
    Open text file named "<currentUser>_booking.txt" for reading.
    If the file is not opened, then:
        Display error message stating user can't check in when no booking is made.
        End function.
    End if

    Close the booking text file.

    If userCurrentstatus's paymentstatus = "Not Paid", then:
        Display error message stating user can't check in when payment is not made.
        End function.
    Else if userCurrentstatus's checkInstatus = "Checked In", then:
        Display error message stating user already checked in.
    Else
        Loop through from index = 0 up to index = noTickets-1:
            Prompt user to enter passenger[index]'s first and last names and passport
            number.
            Update the currentUserstatus's passengerdetails after each input.
            If index > 0, then:
                Prompt user to confirm if passenger[index] has the same contact
                person as previous passenger(s).

                If confirm = yes, then:

```

```

        If index > 1, then:
            Prompt user to enter the index of passenger whose
            the contact person is the same.
            Store the index into variable idx.
        Else
            idx = 0.
        End if

        Store passenger[idx]'s contact person's first and last names
        and mobile phone number into passenger[index]'s contact
        person's details.
        Proceed to next loop.
    End if
End if

    Prompt user to enter passenger[index]'s contact person's first and last names
    and mobile phone number.
    Update the currentUserstatus's passengerdetails after each input.
End loop

Prompt user on confirmation for check in.

If confirm = no, then:
    End function.
Else
    Update currentUserstatus checkInstatus = "Checked In".
    Open text file named "<currentUser>_paymentCheckin.txt" for writing.
    If the file is opened, then:
        Write the currentUserstatus's total ticket price and payment status
        into the file.
        Loop through index = 0 to index = noTickets-1:
            Write the currentUserstatus's passenger details into the file.
        End loop
        Write the currentUserstatus's check in status into the file.
    End if
    Close the paymentcheckin text file.
End if
End if
End function.

```

Define printInvoice(bookingDetails flight, passengerStatus passenger)

Start

Open text file named "<currentUser>_booking.txt" for reading.

If the file is not opened, then:

Display error message stating no booking is made.

End function.

Else if the current user status's payment status = "Not Paid", then:

Display error message stating the user has not made payment.

End function.

End if

Close the booking text file.

Open the text file named "<currentUser>_invoice.txt" for writing.

Write the current user's departure and return flight routes, date and time in to the file.

Loop through index = 0 to index = noTickets-1:

Write passenger[index+1] in to the file

Write currentUserstatus's passenger[index]'s first and last names, passport number, contact person first and last names and mobile phone number.

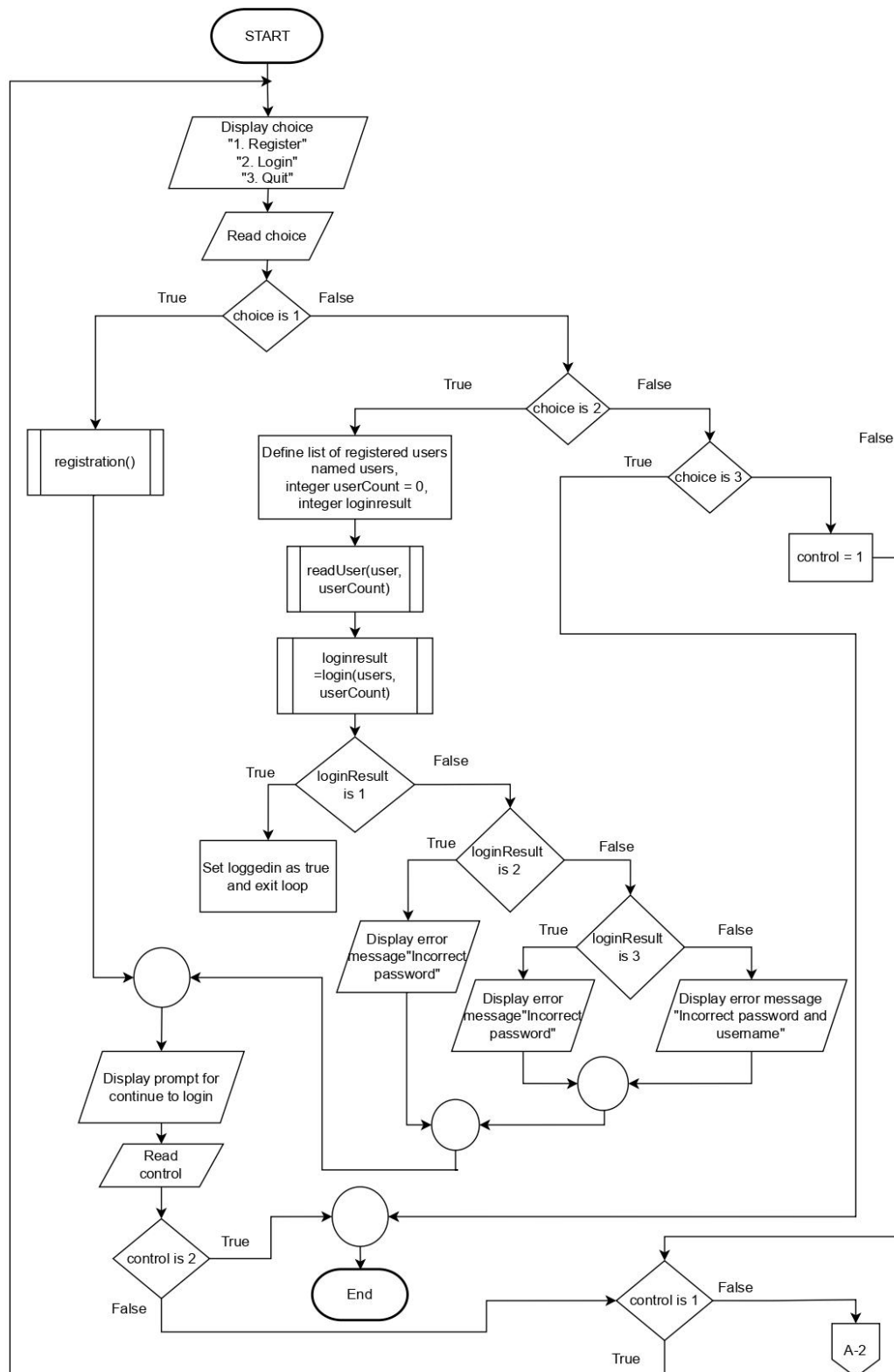
End loop

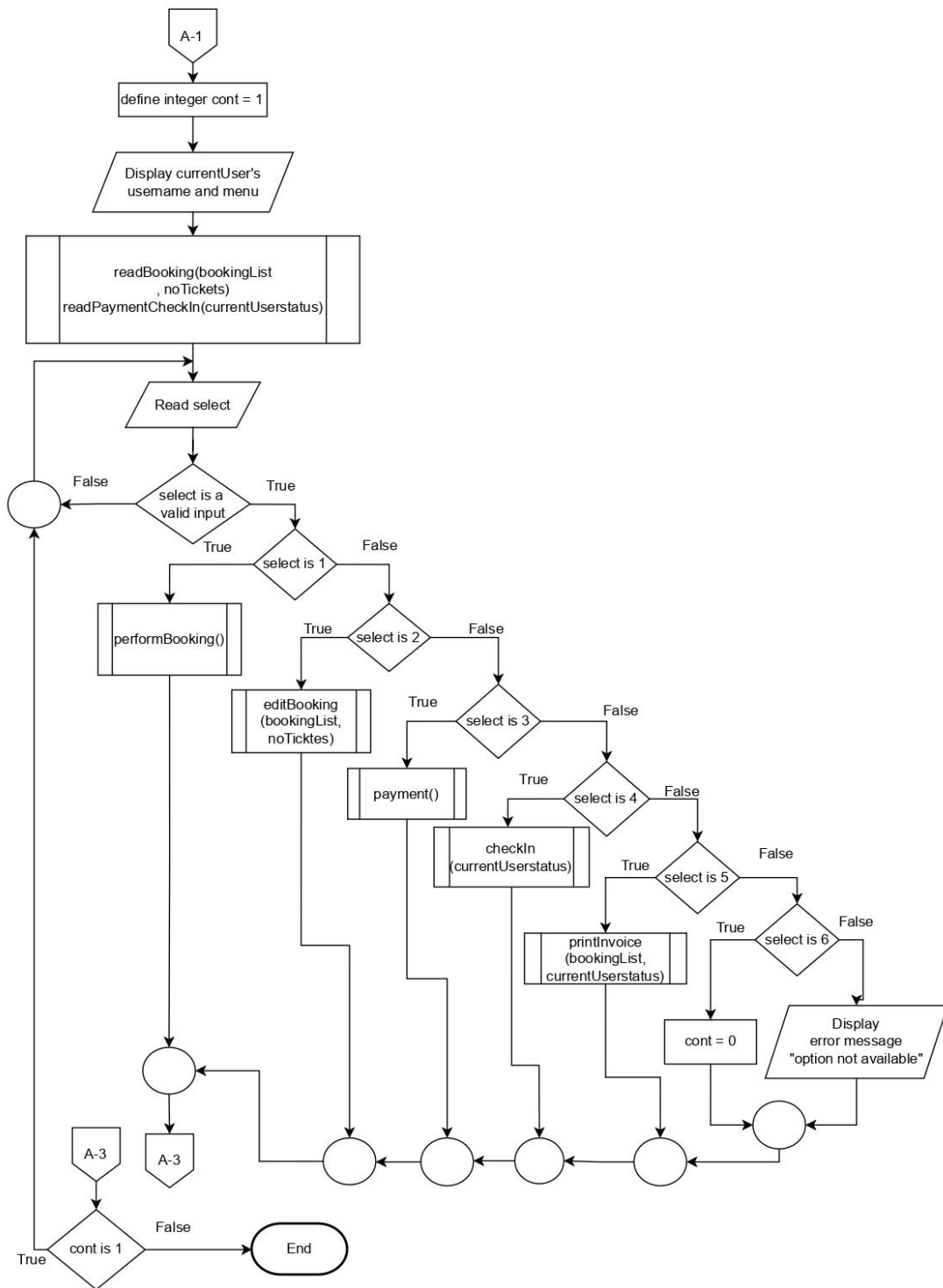
Close the invoice text file.

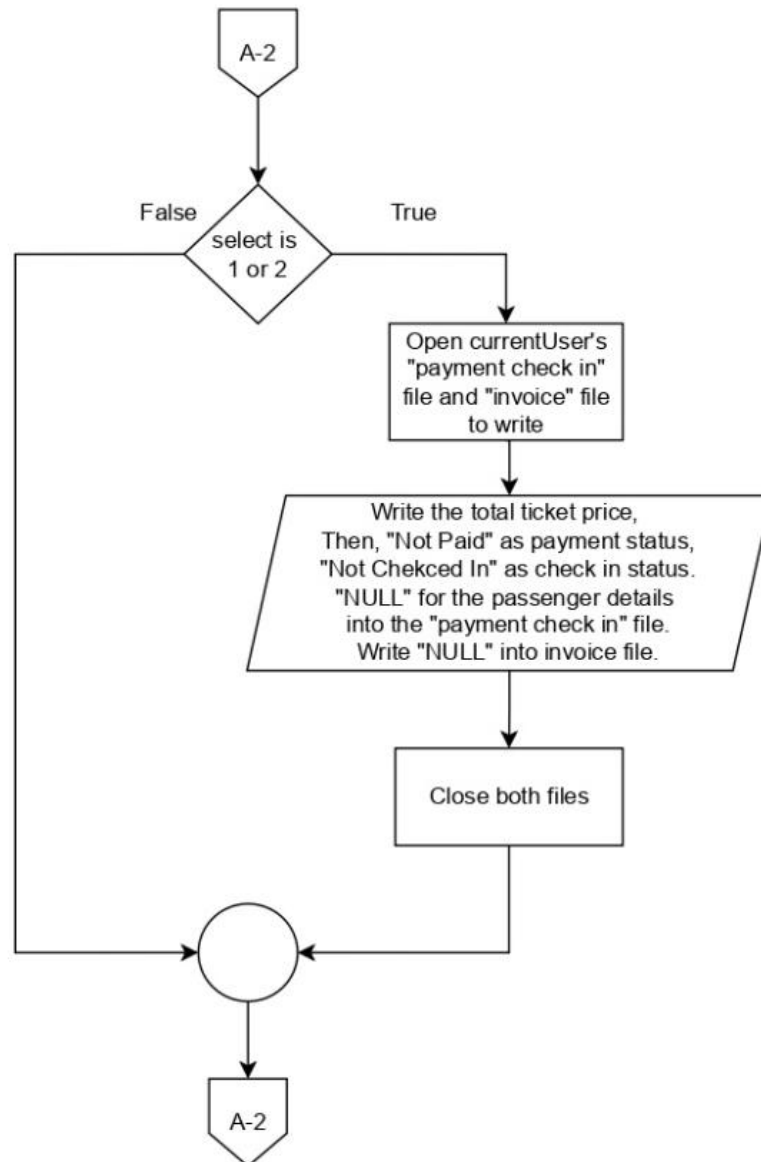
Display message stating the invoice has been printed.

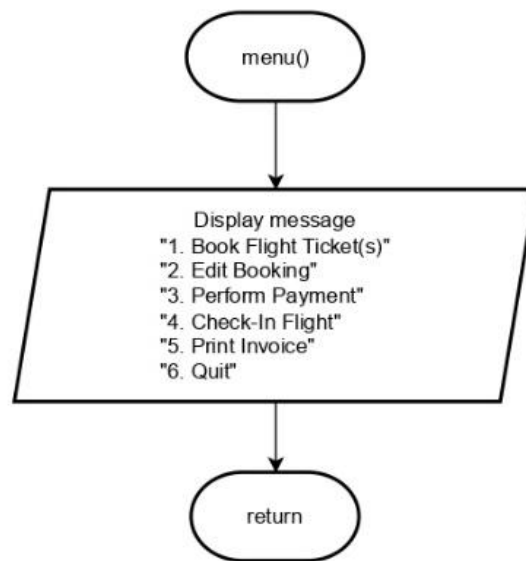
End function

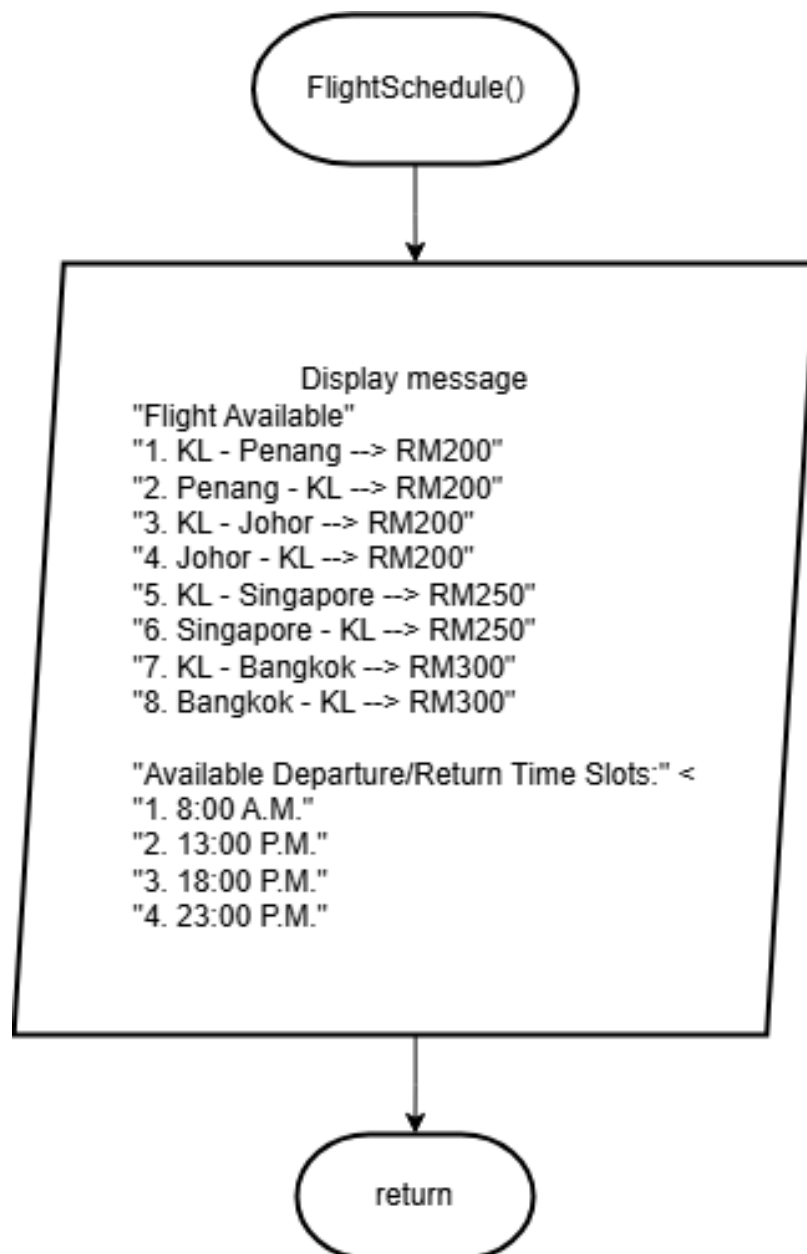
Flowchart

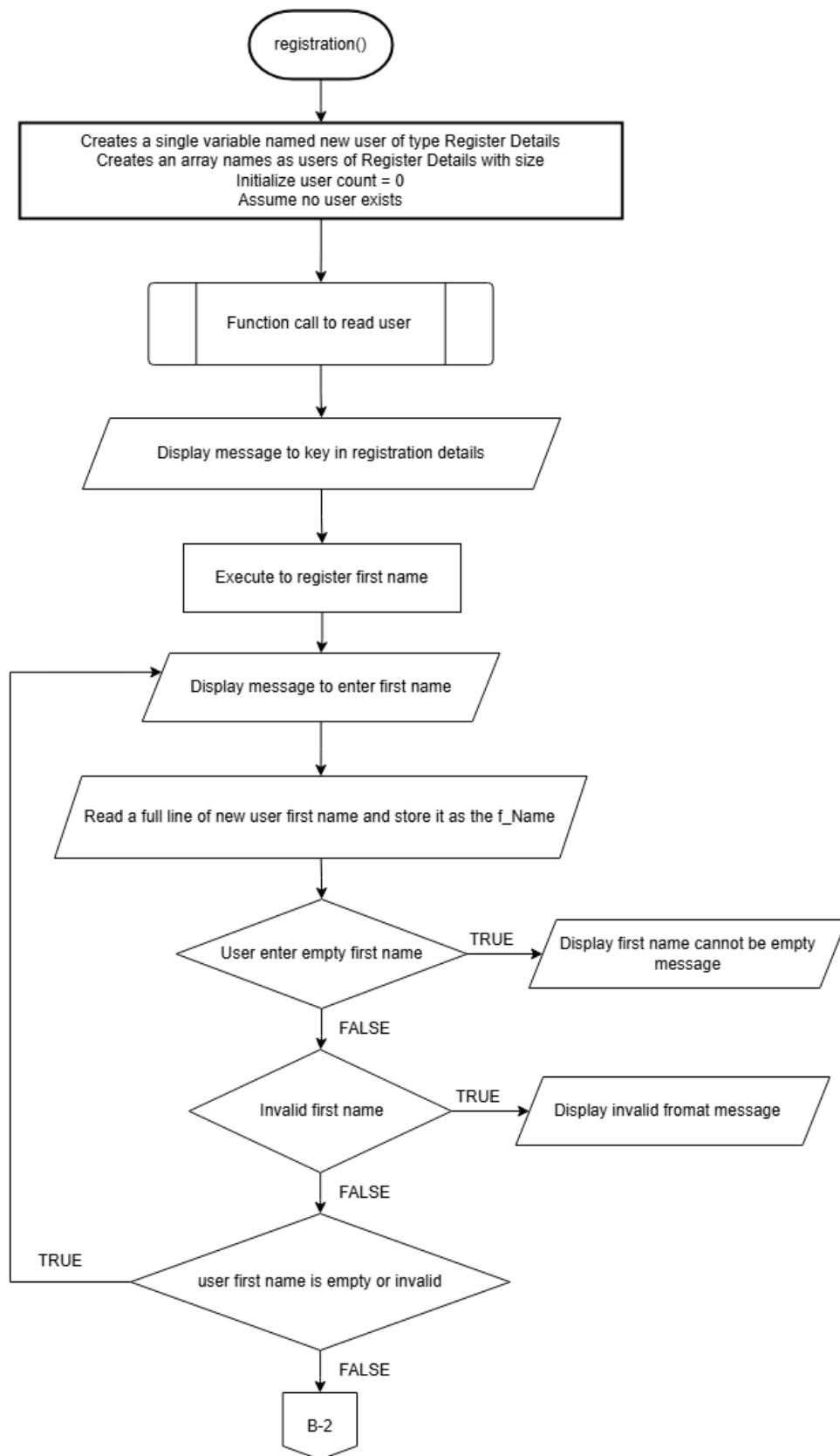


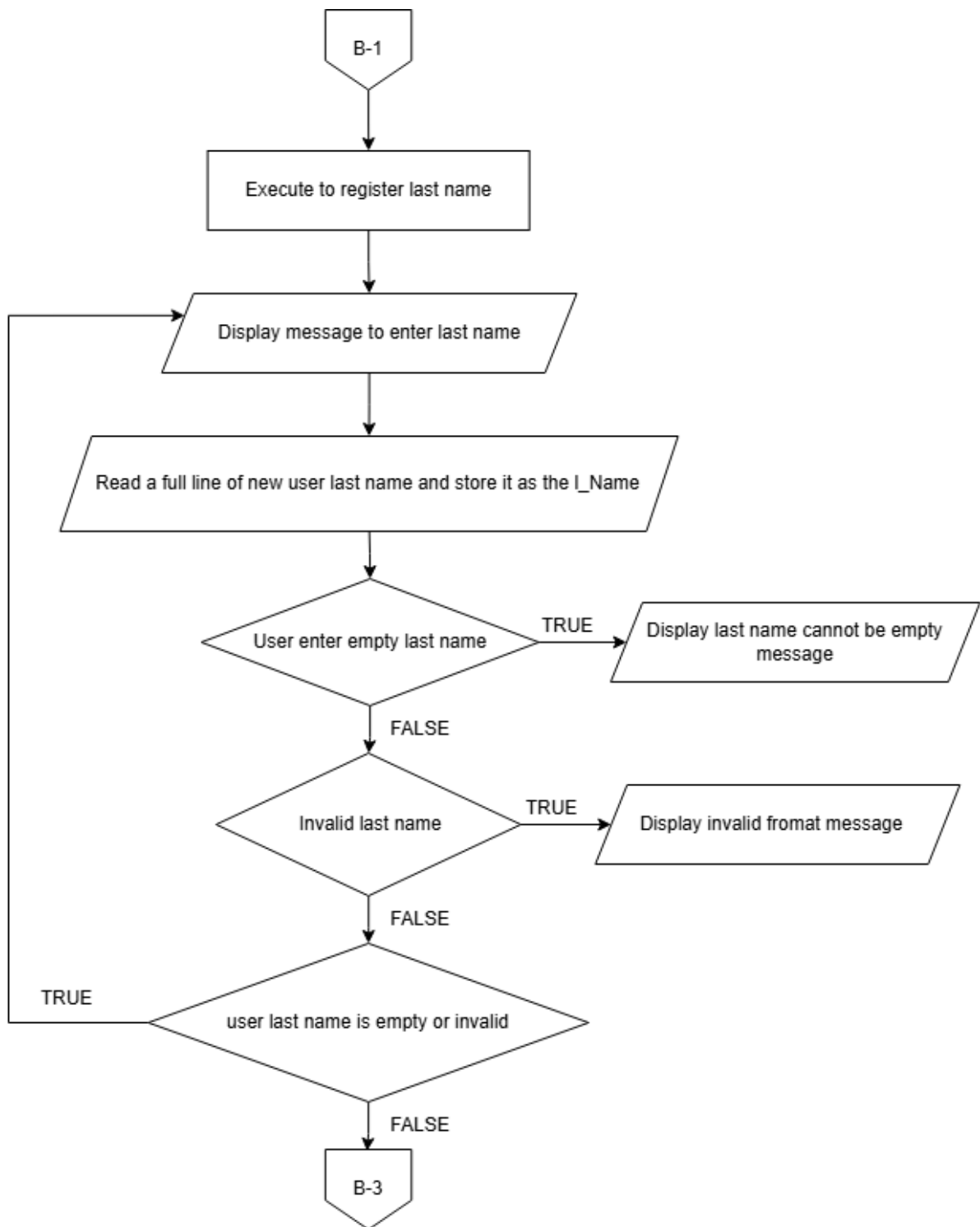


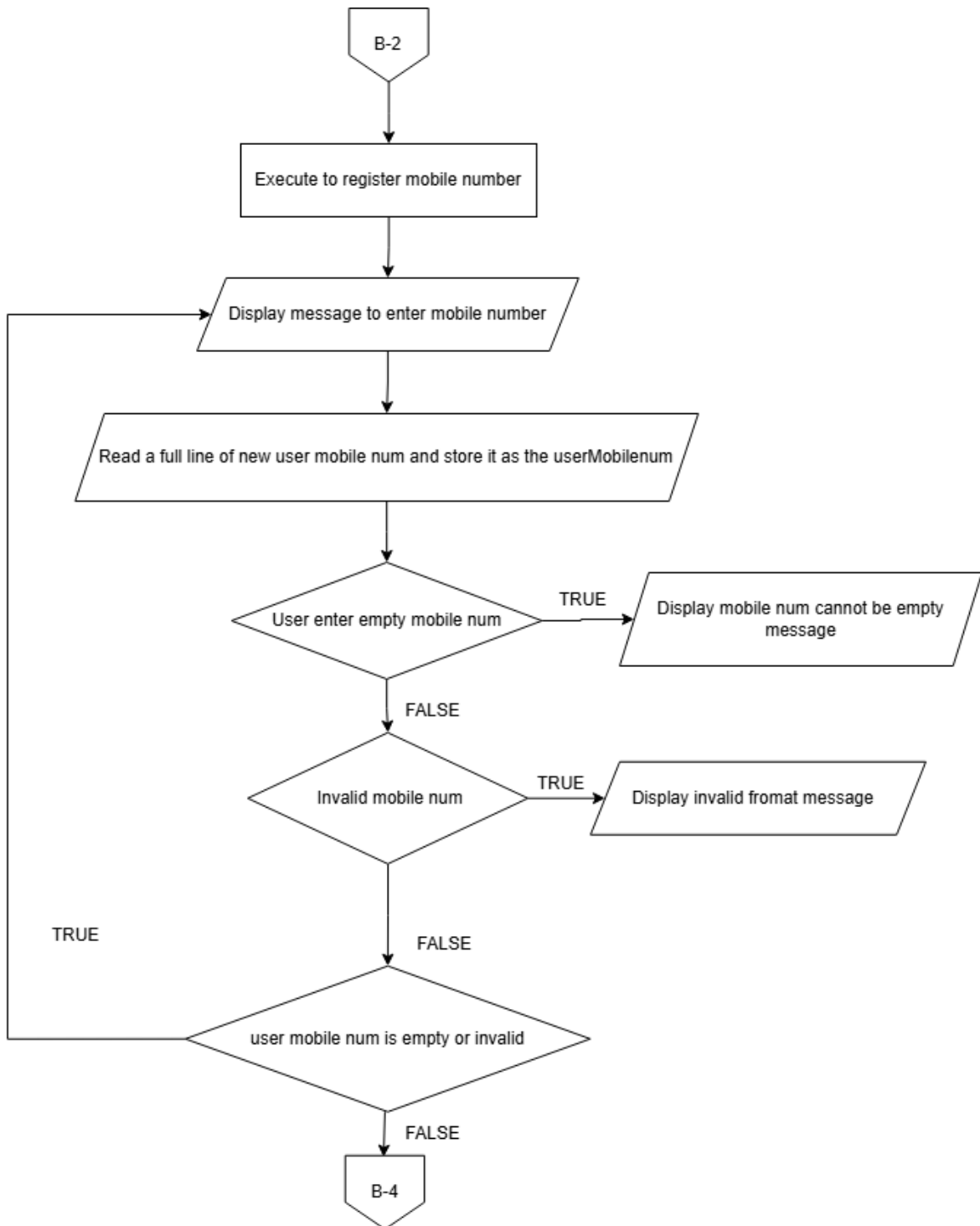


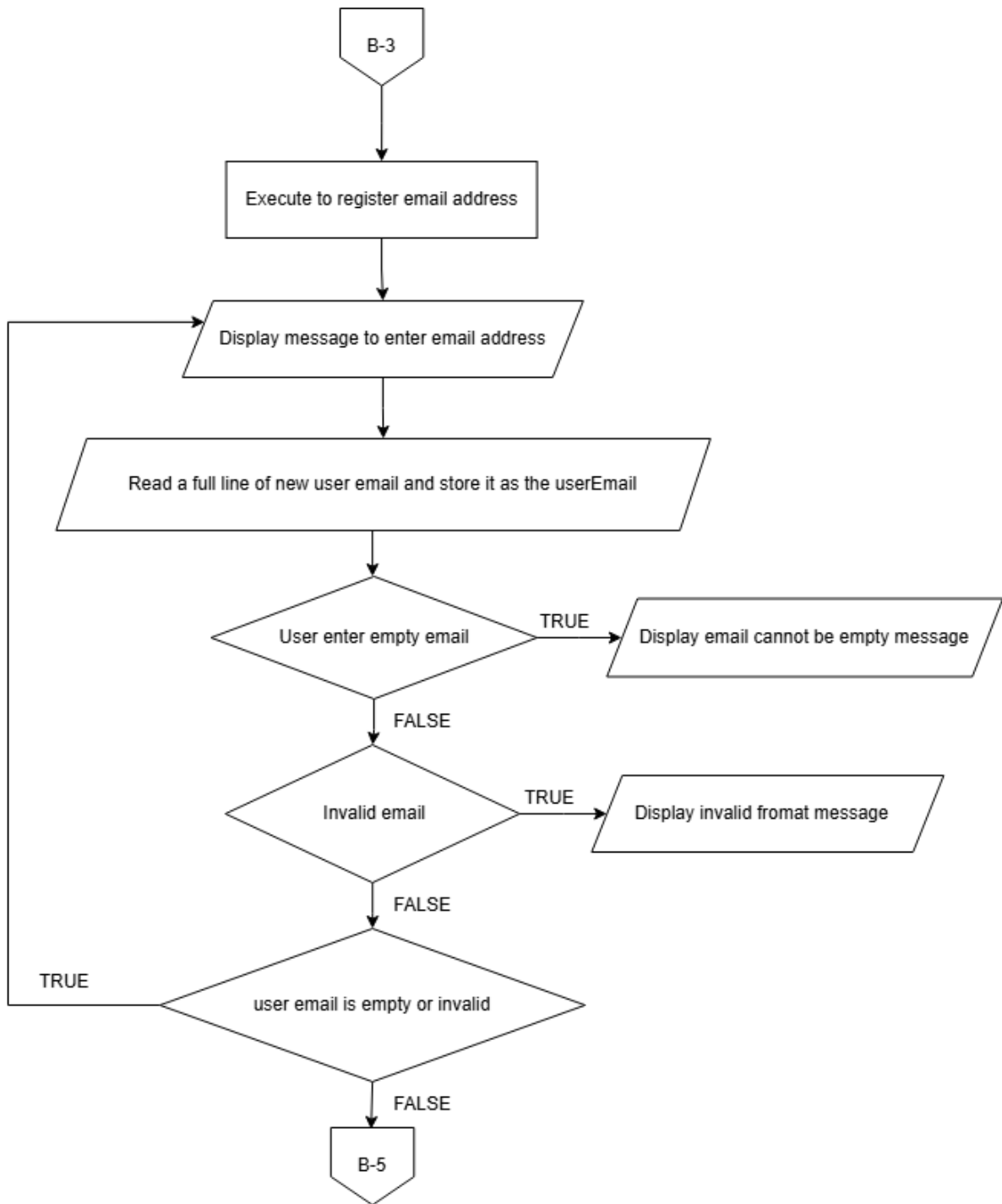


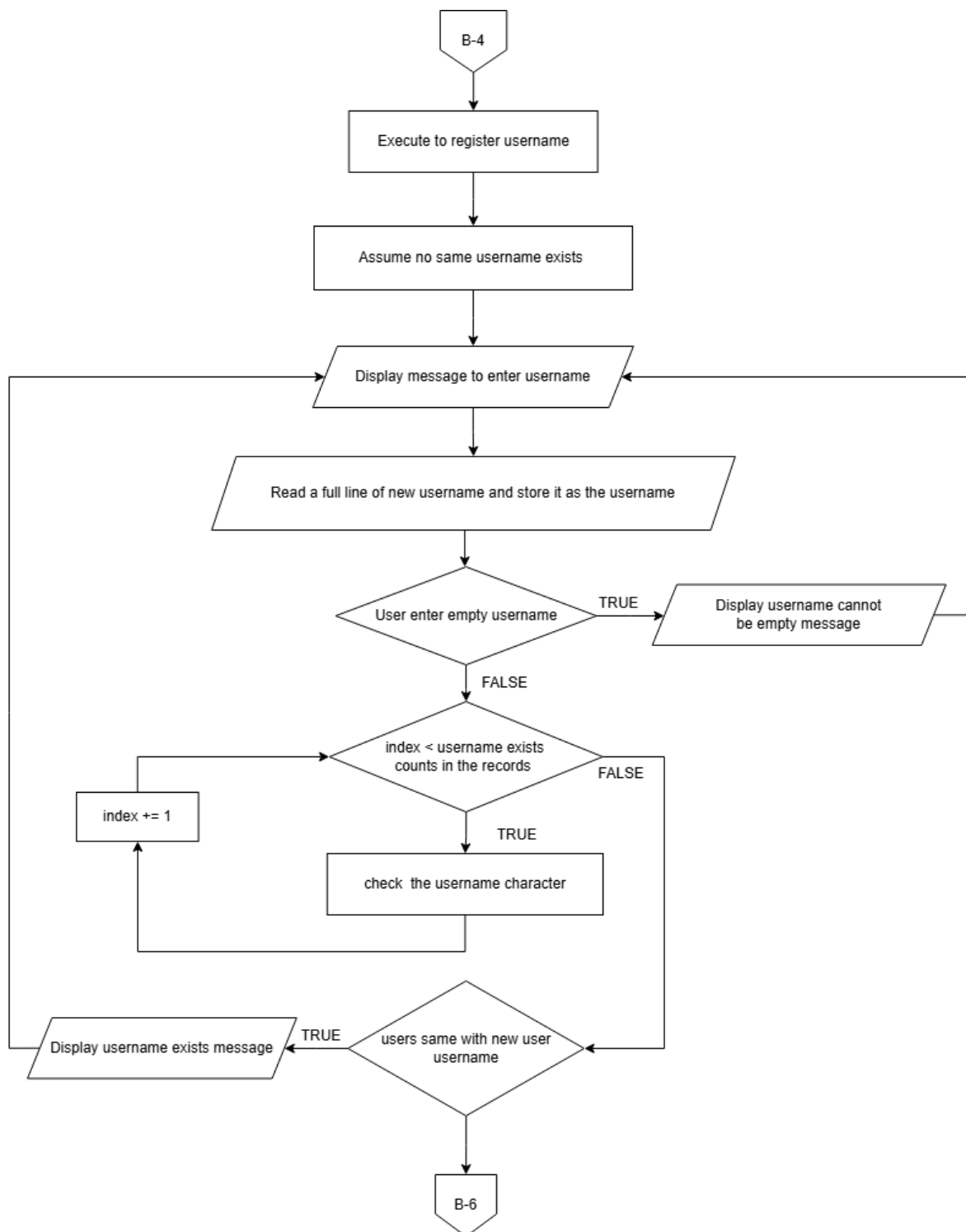


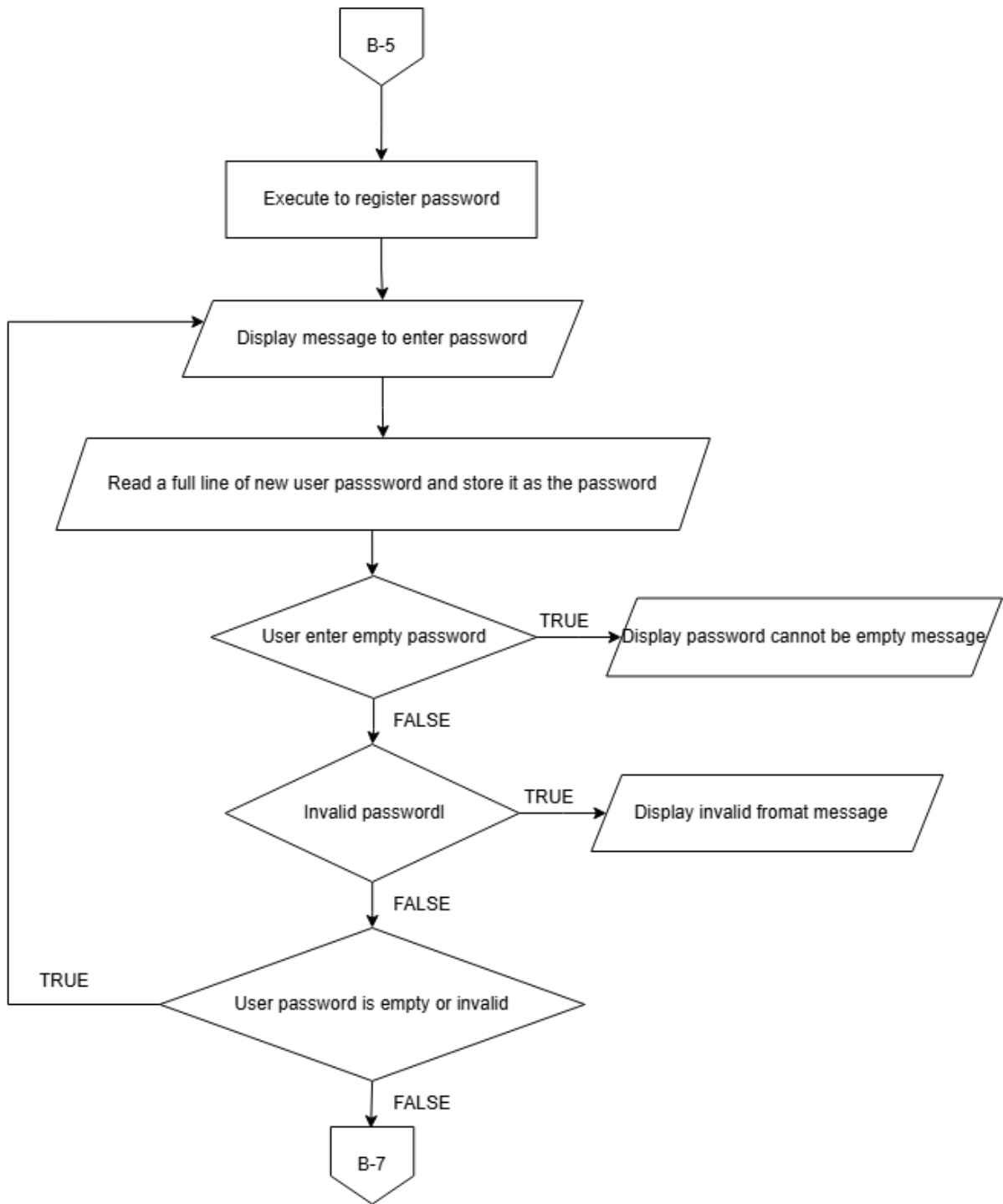


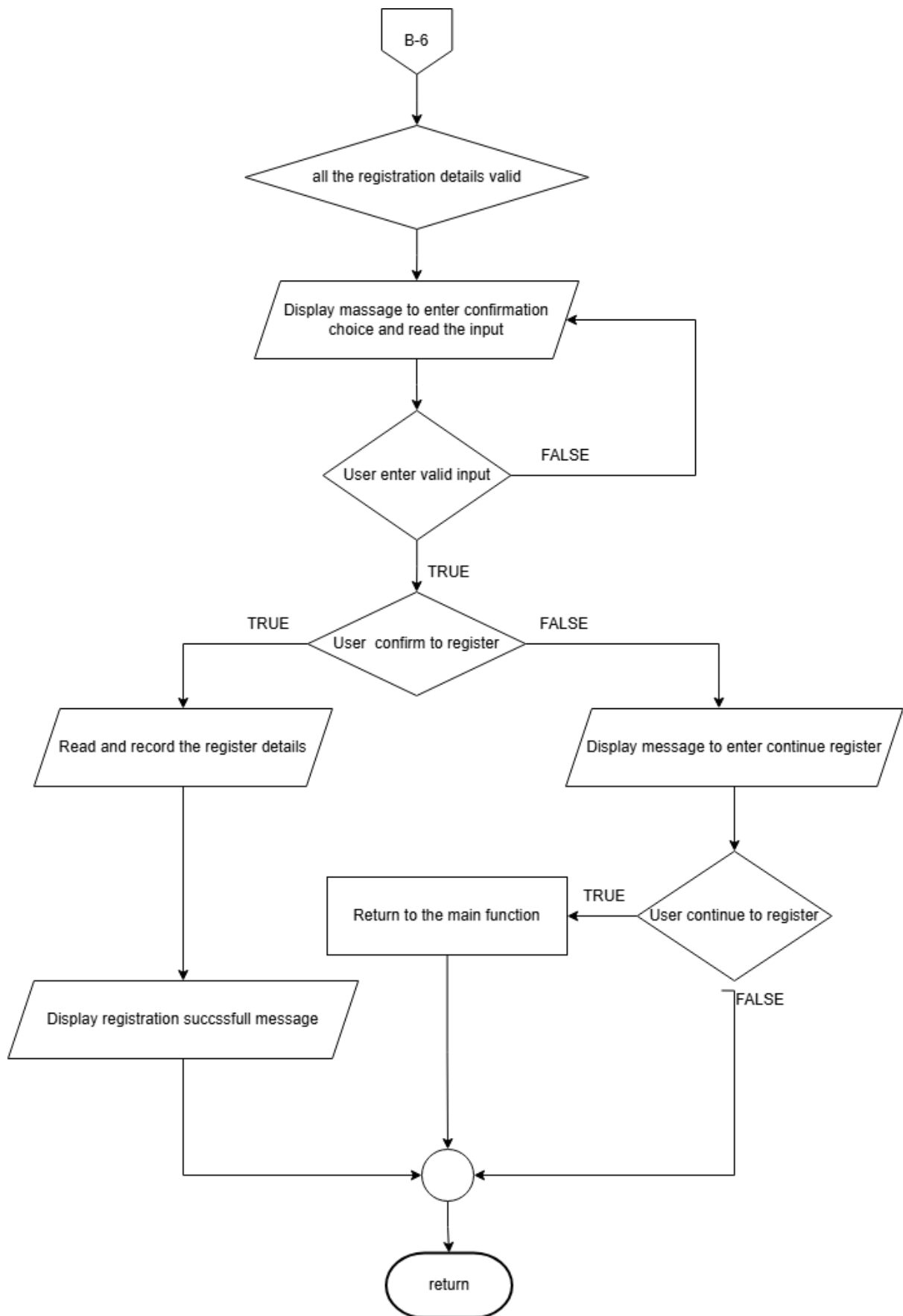


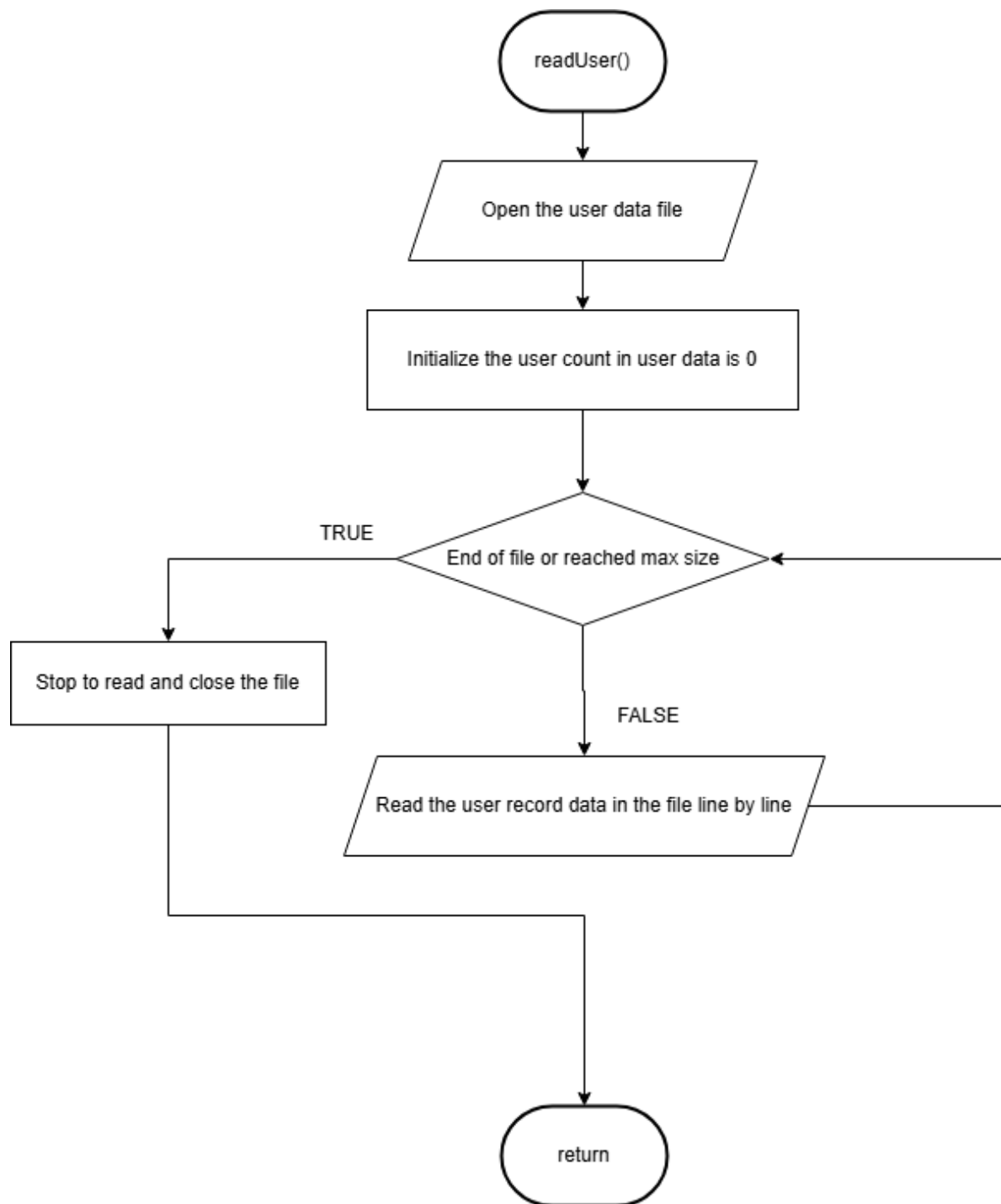


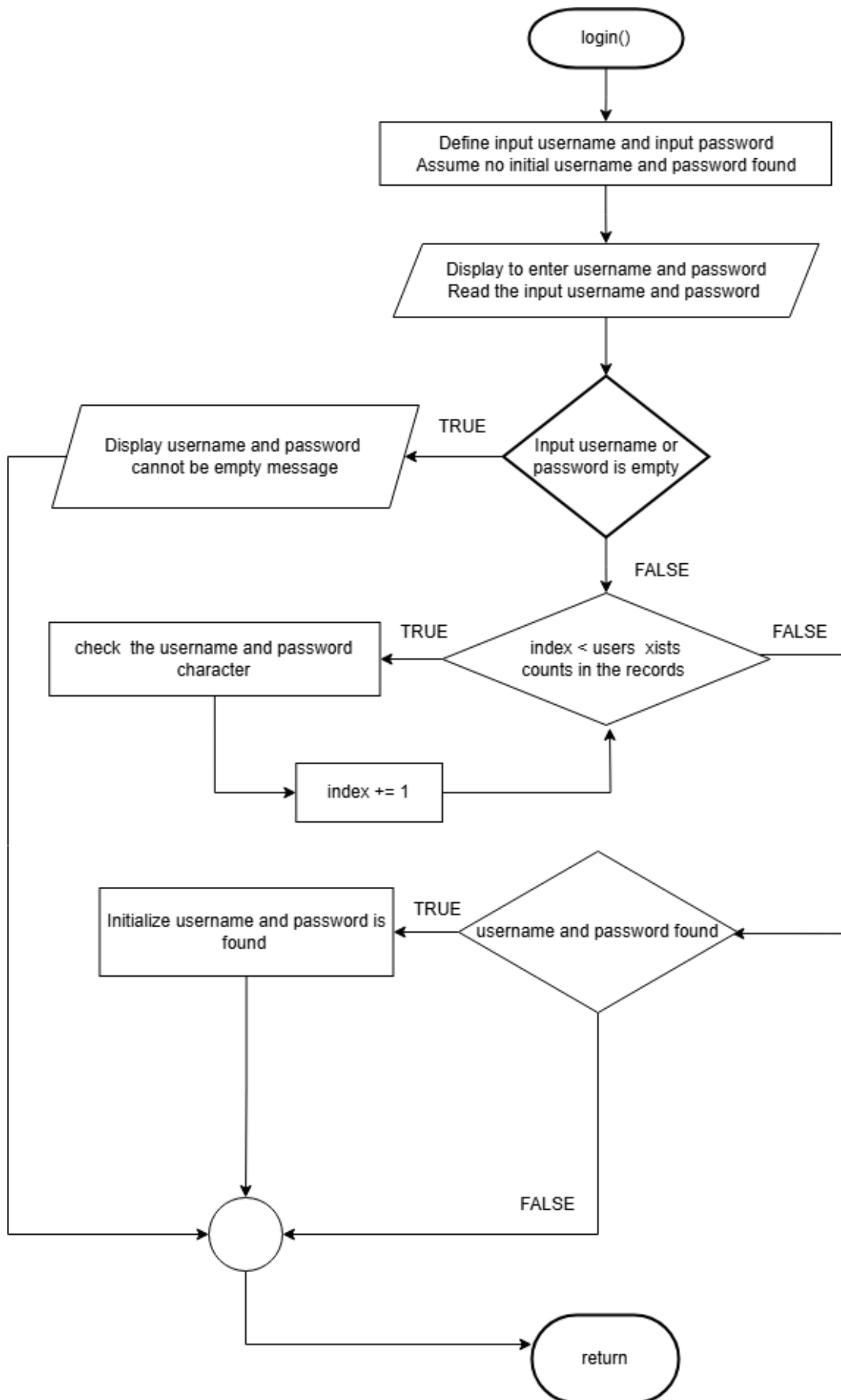


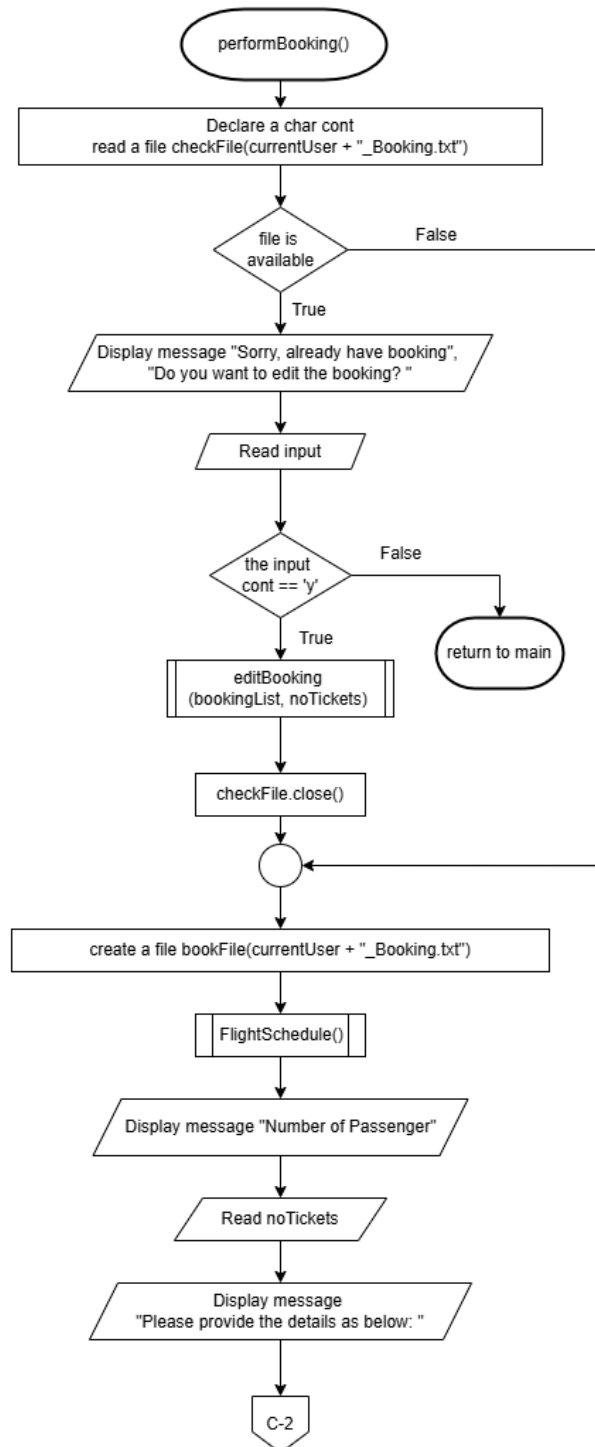


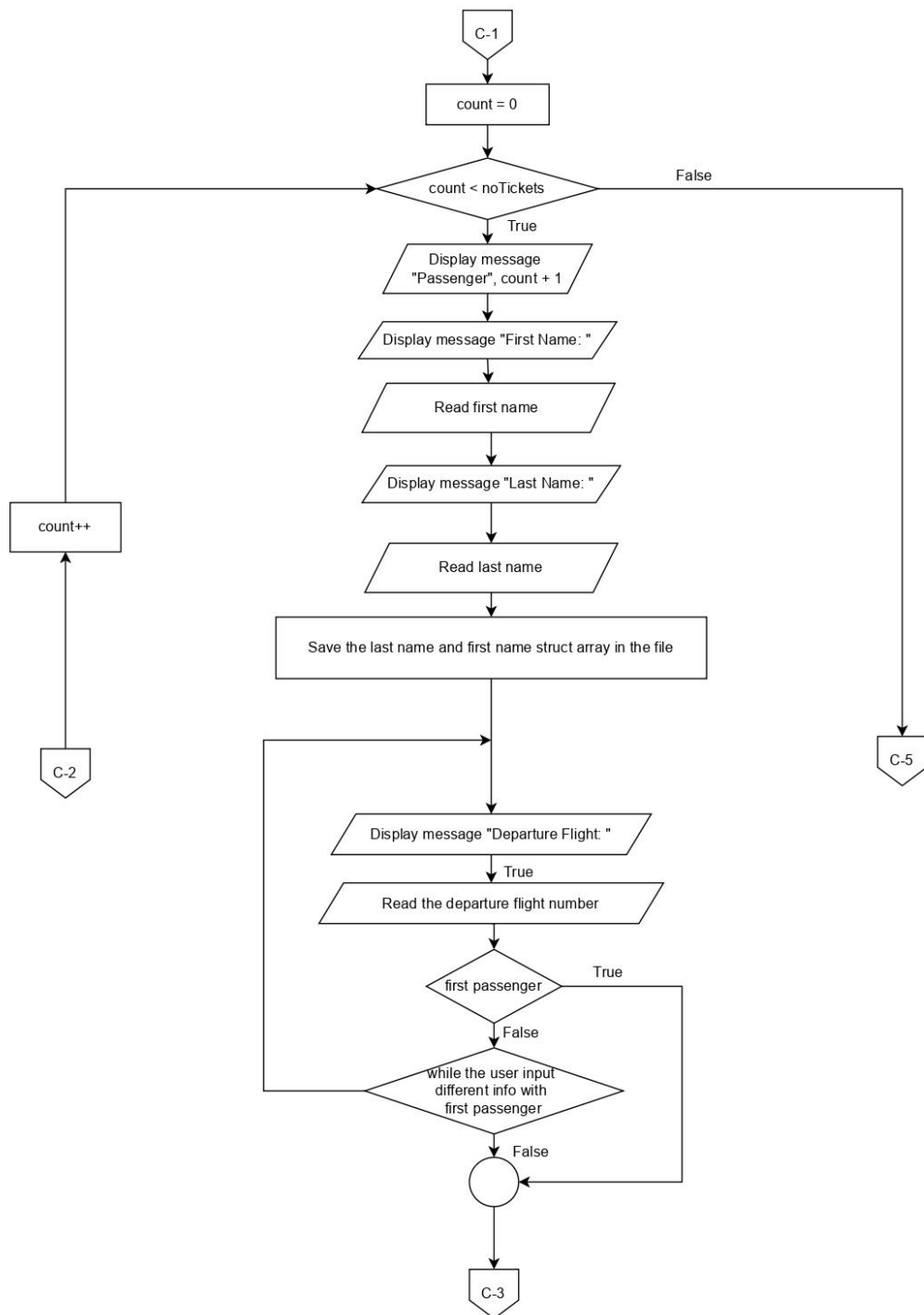


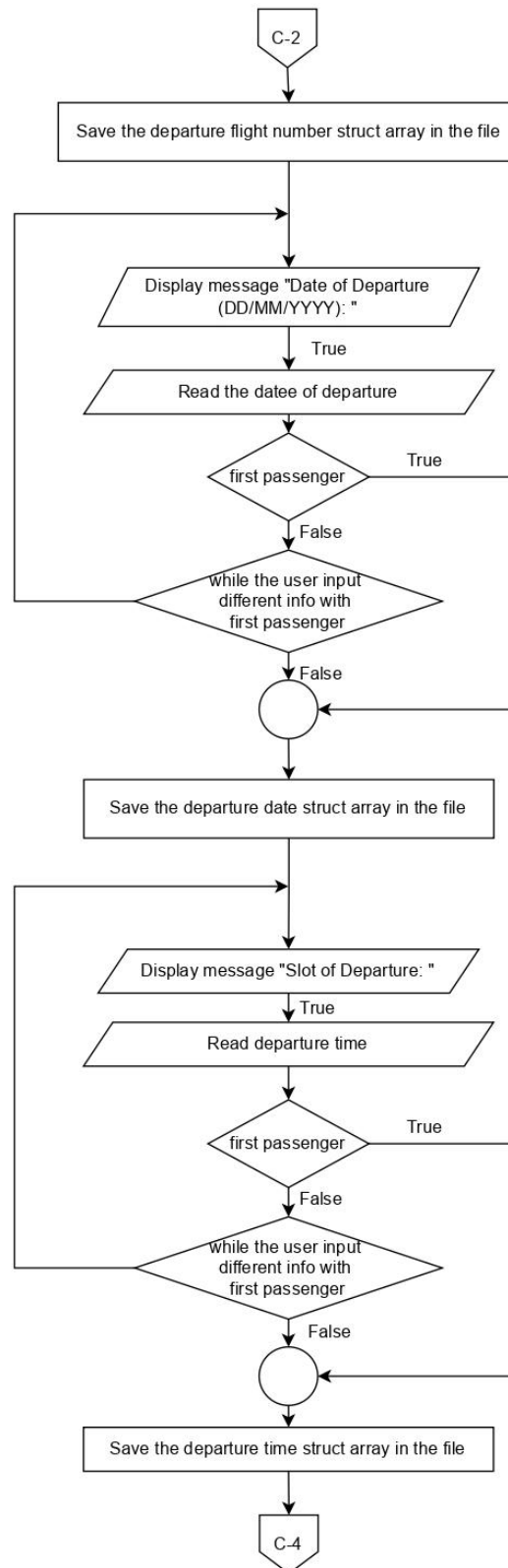


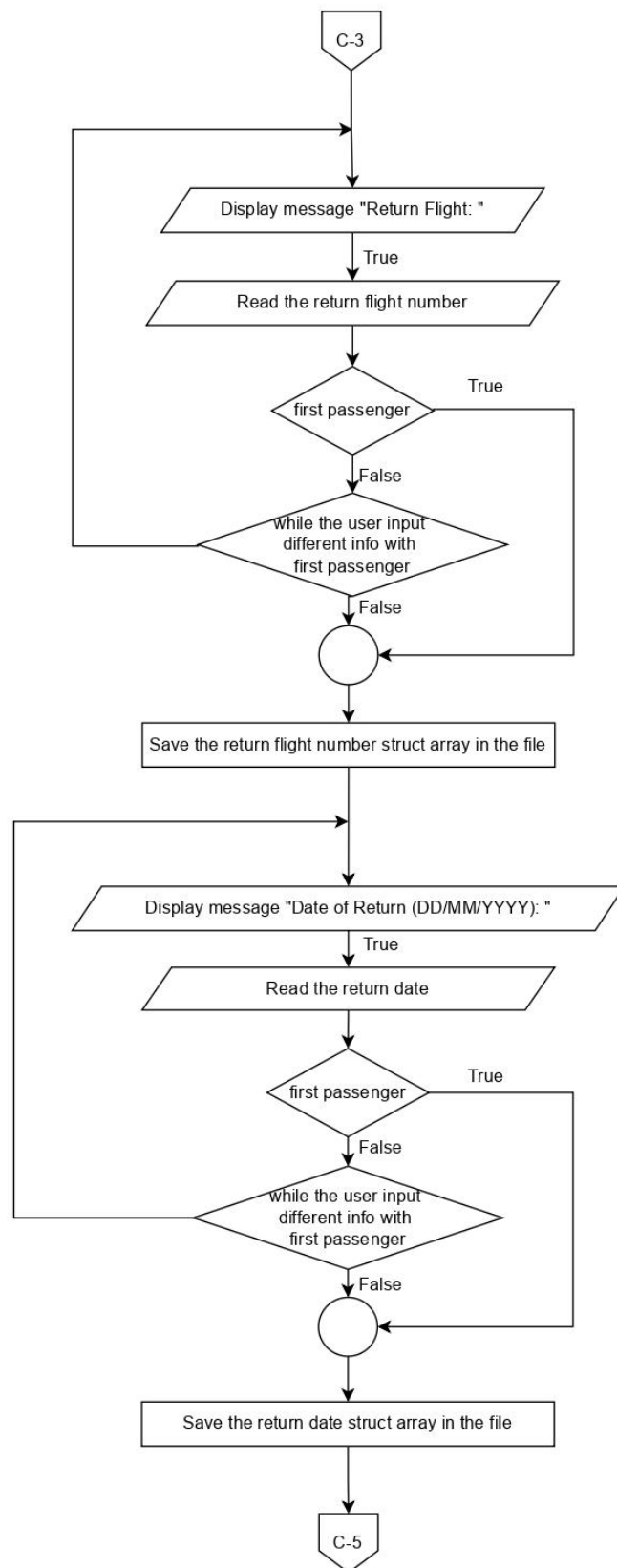


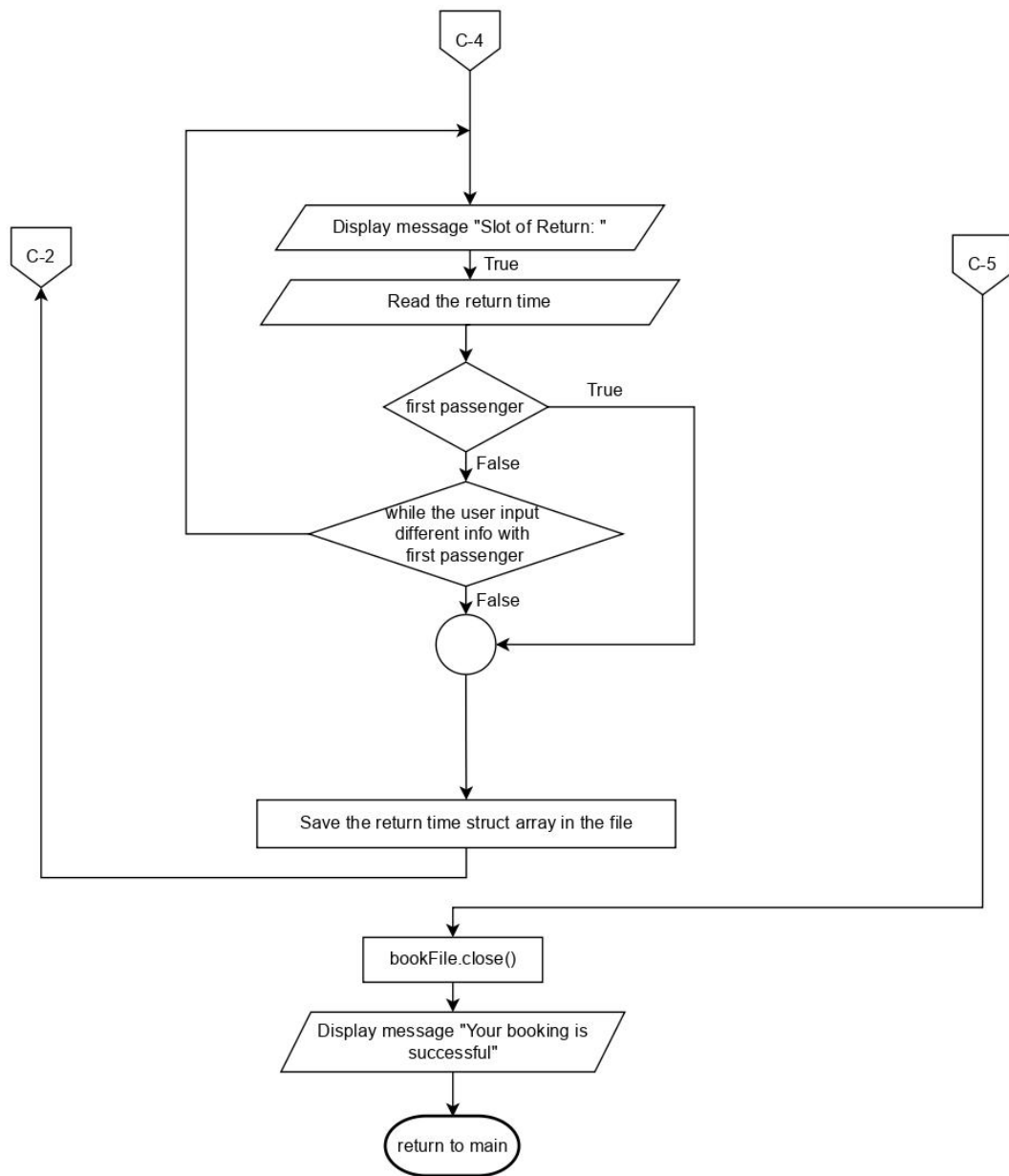


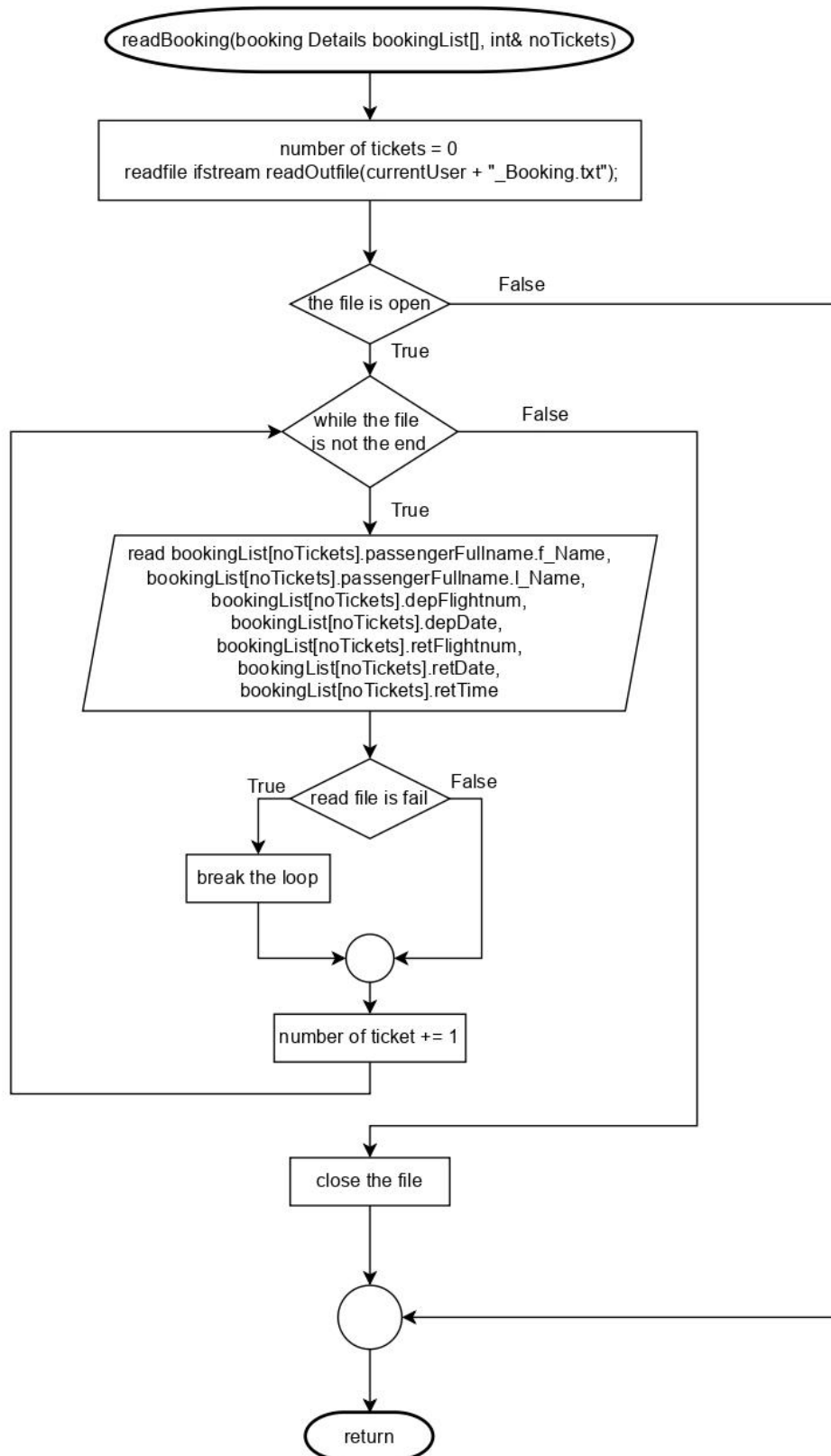


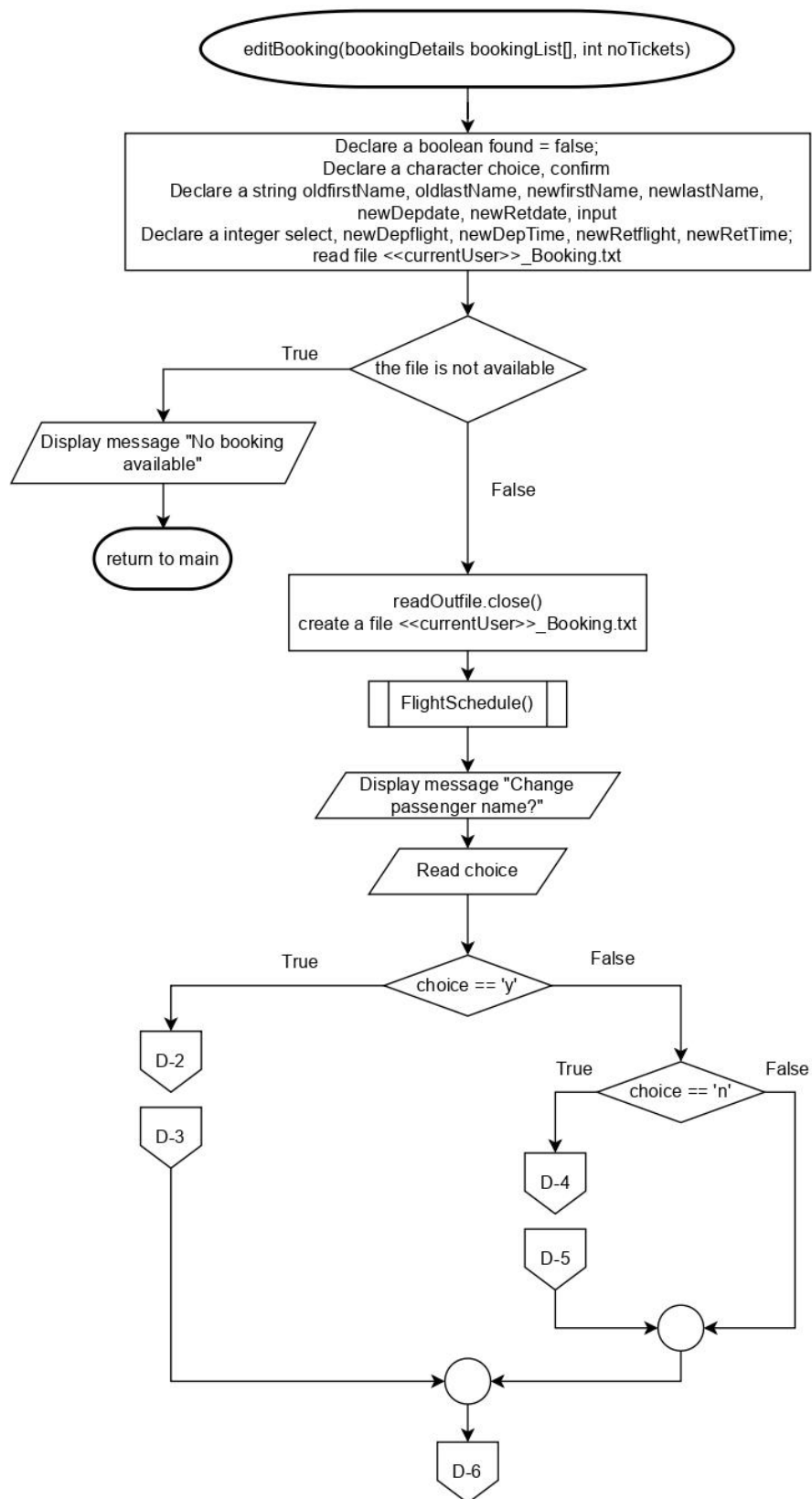


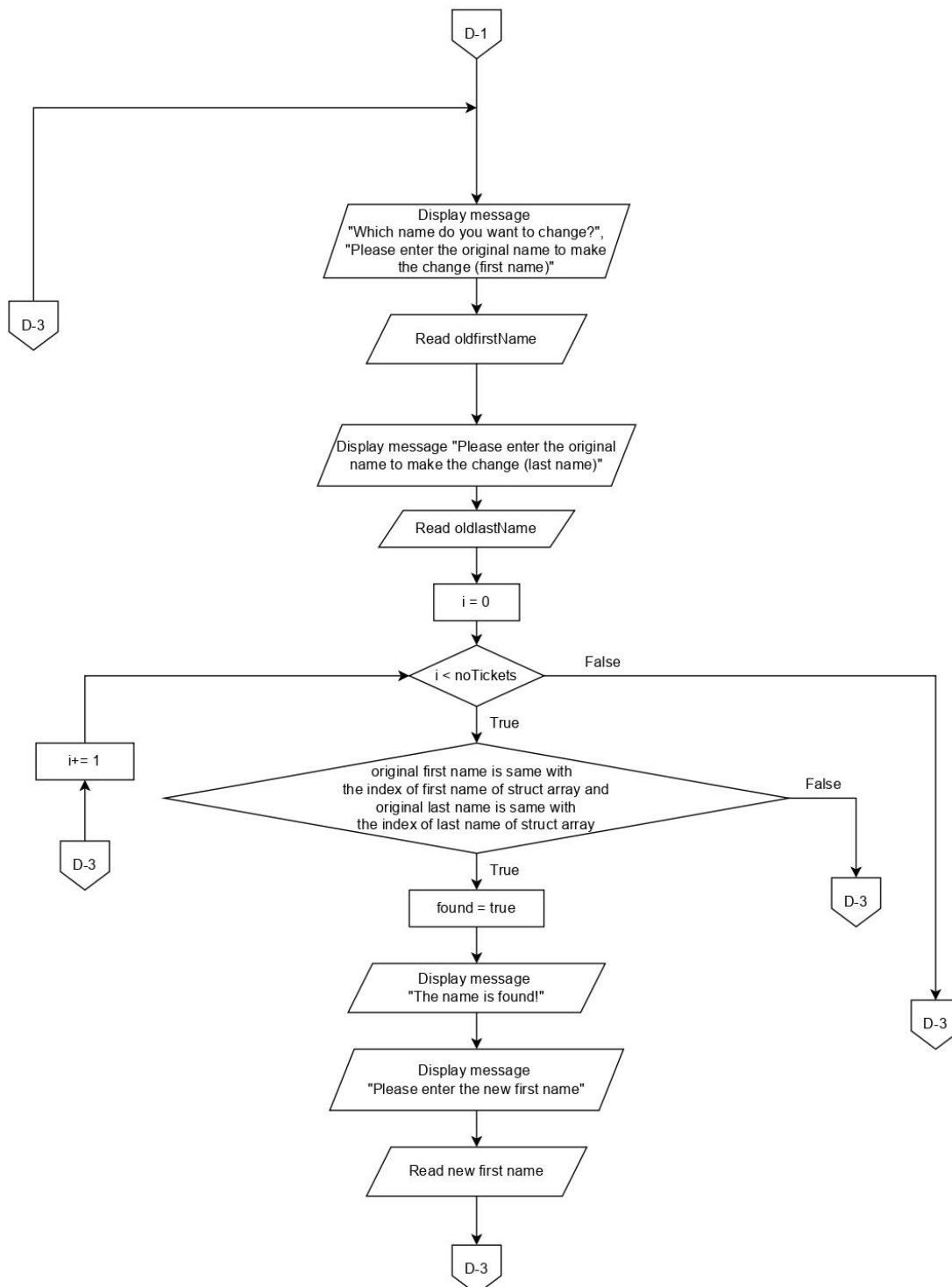


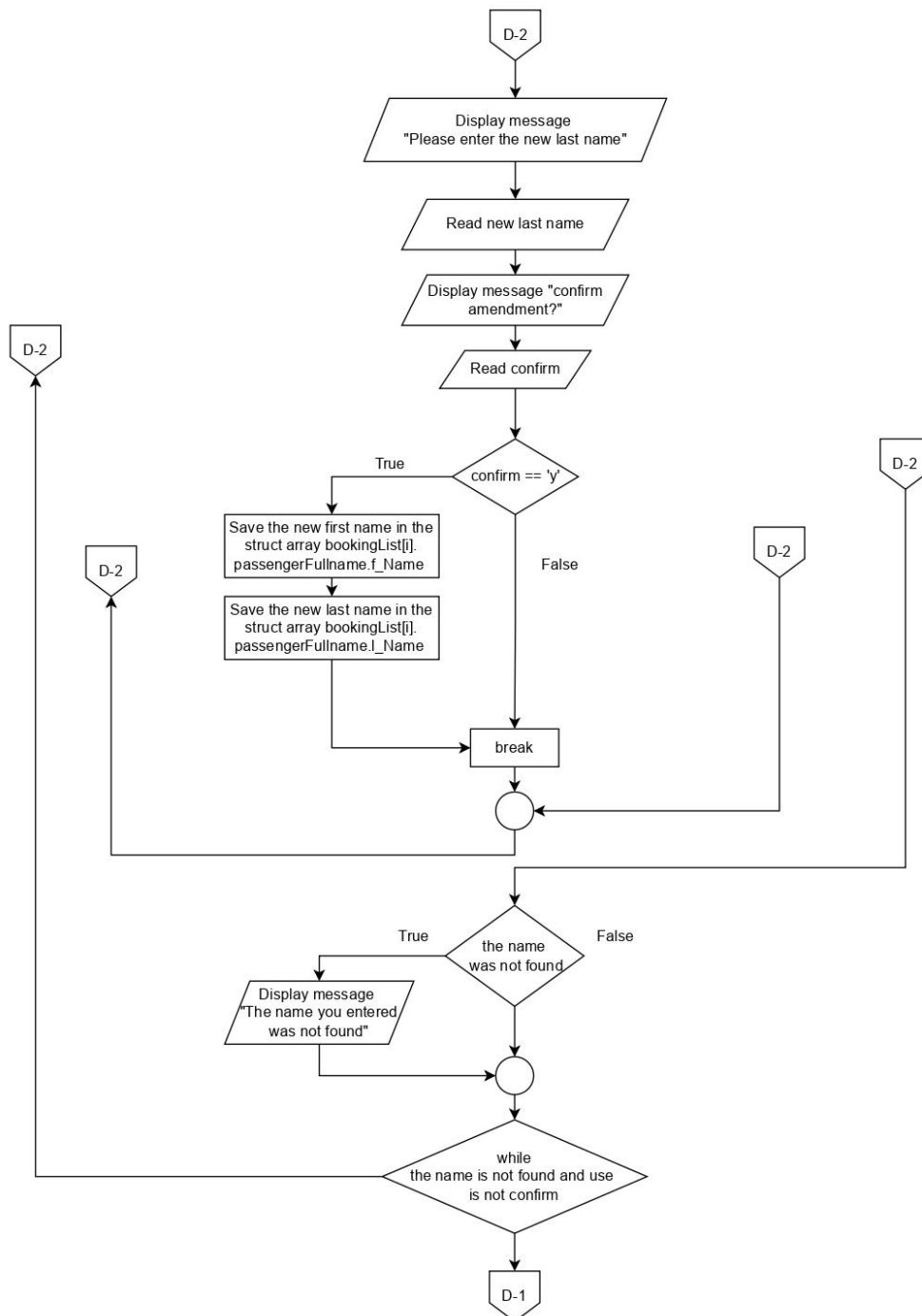


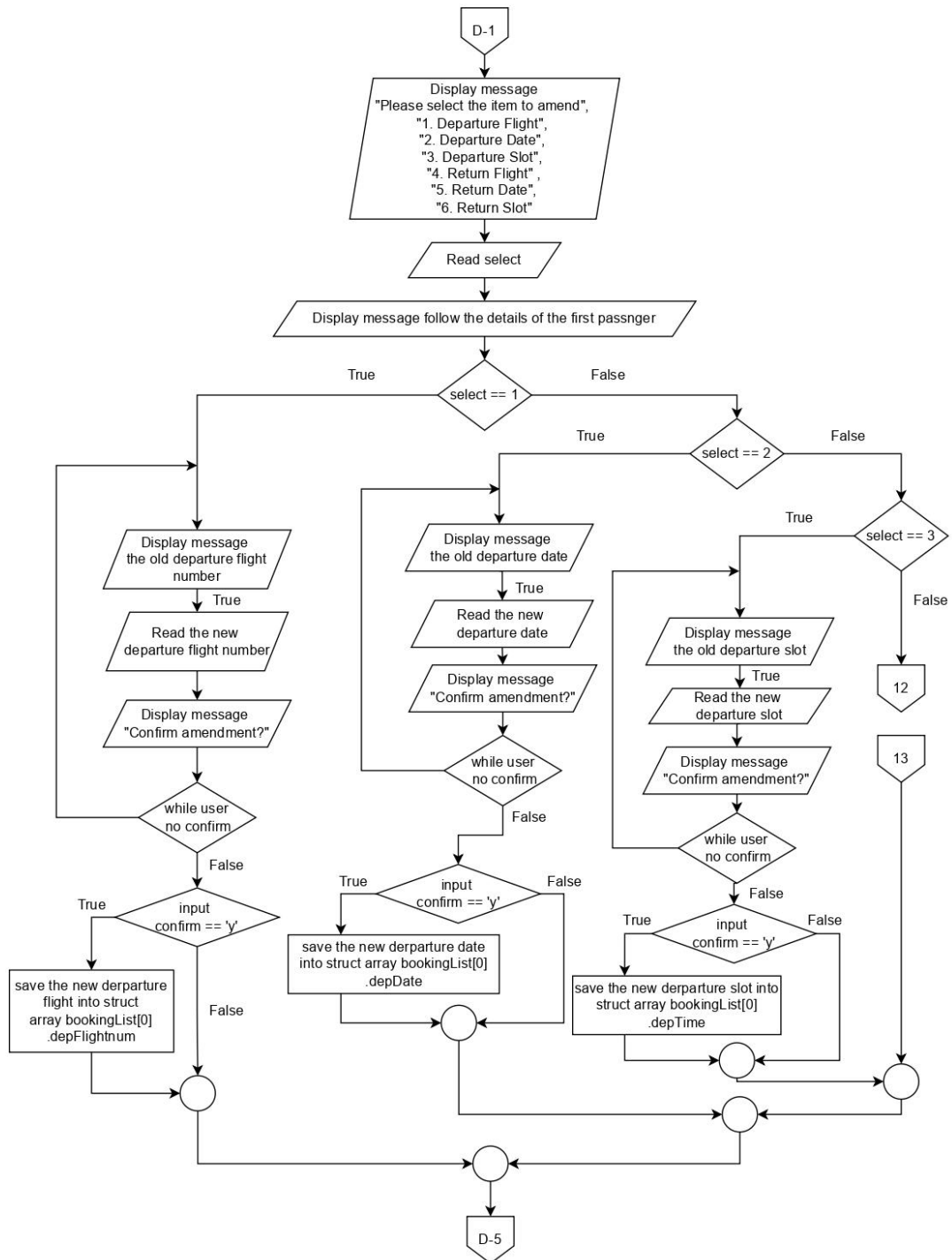


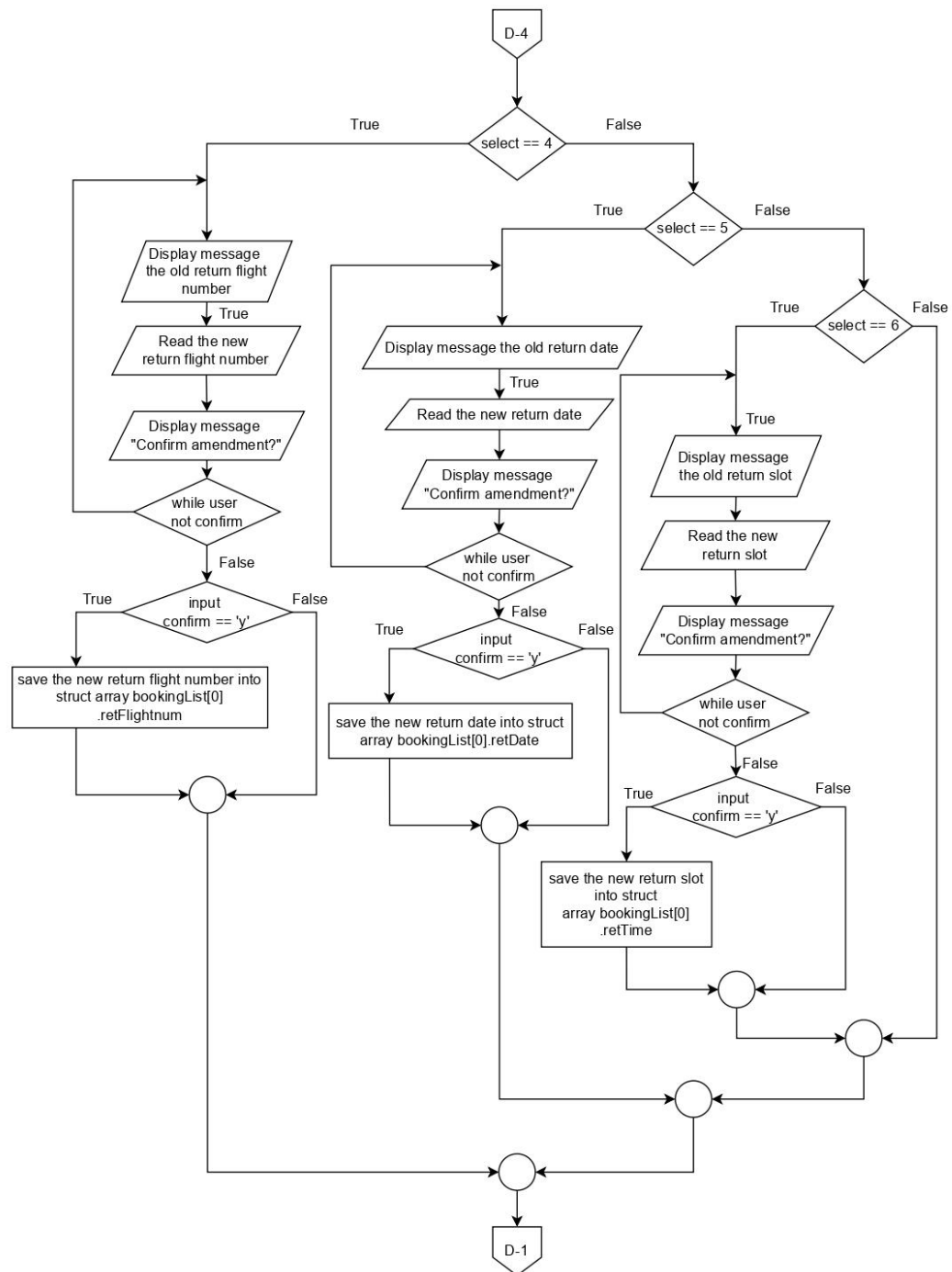


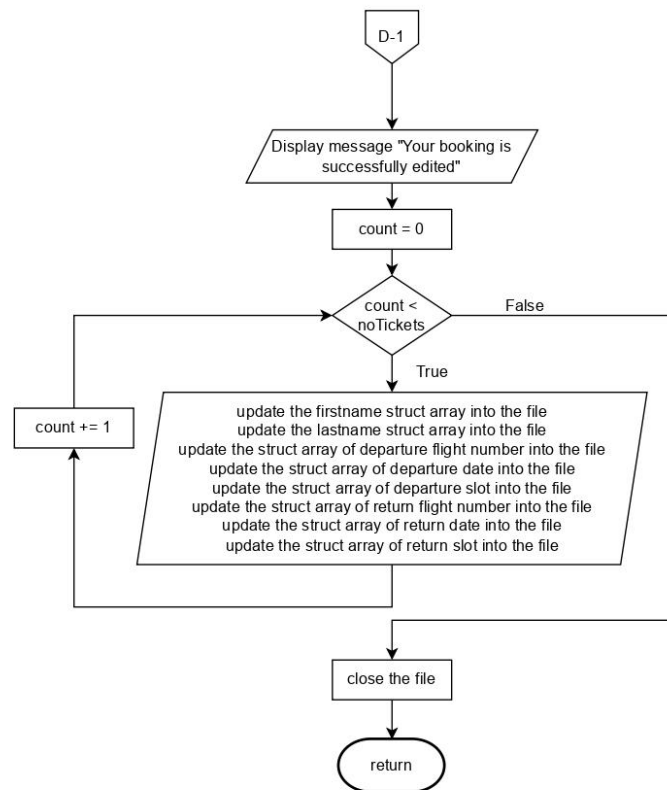


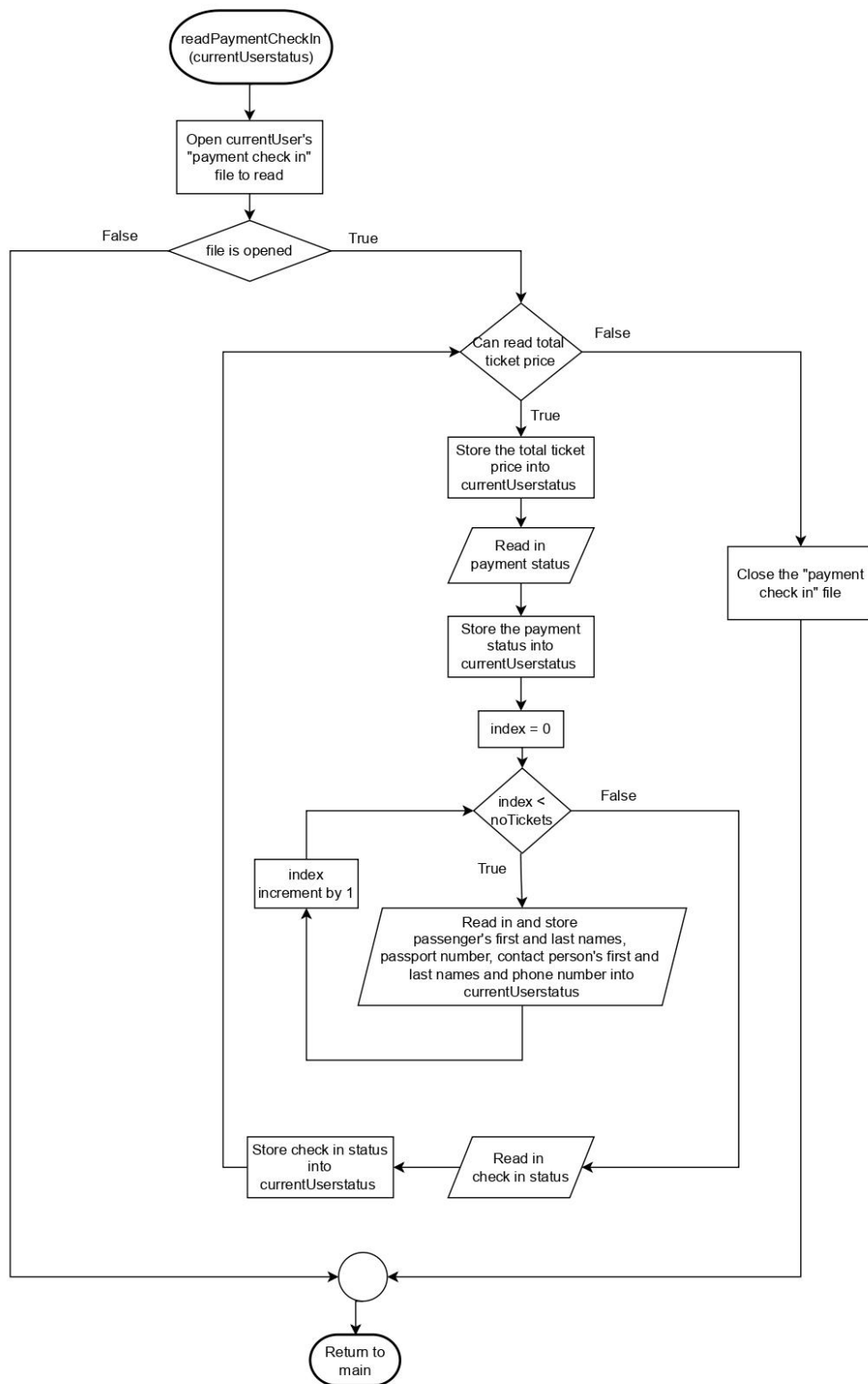


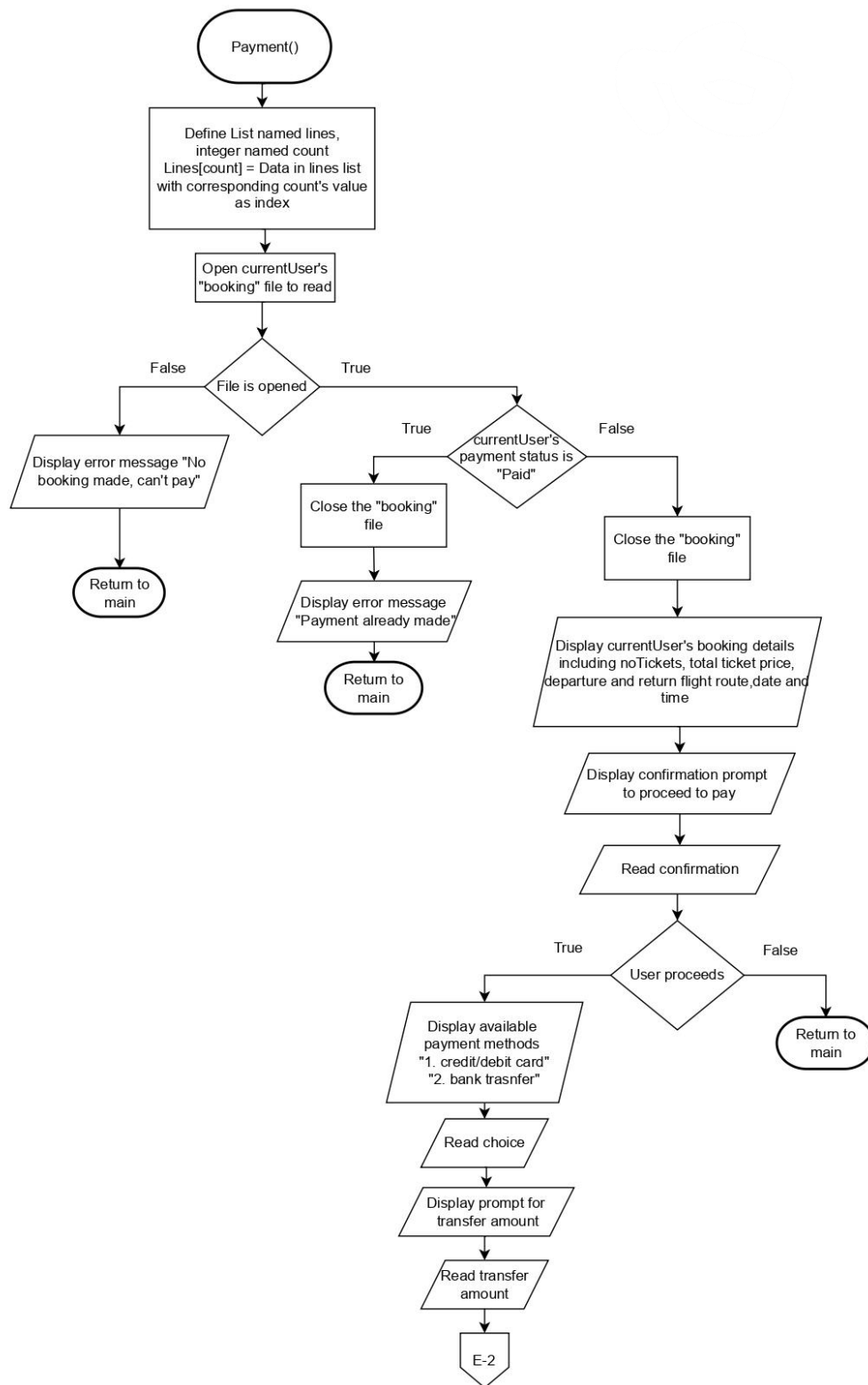


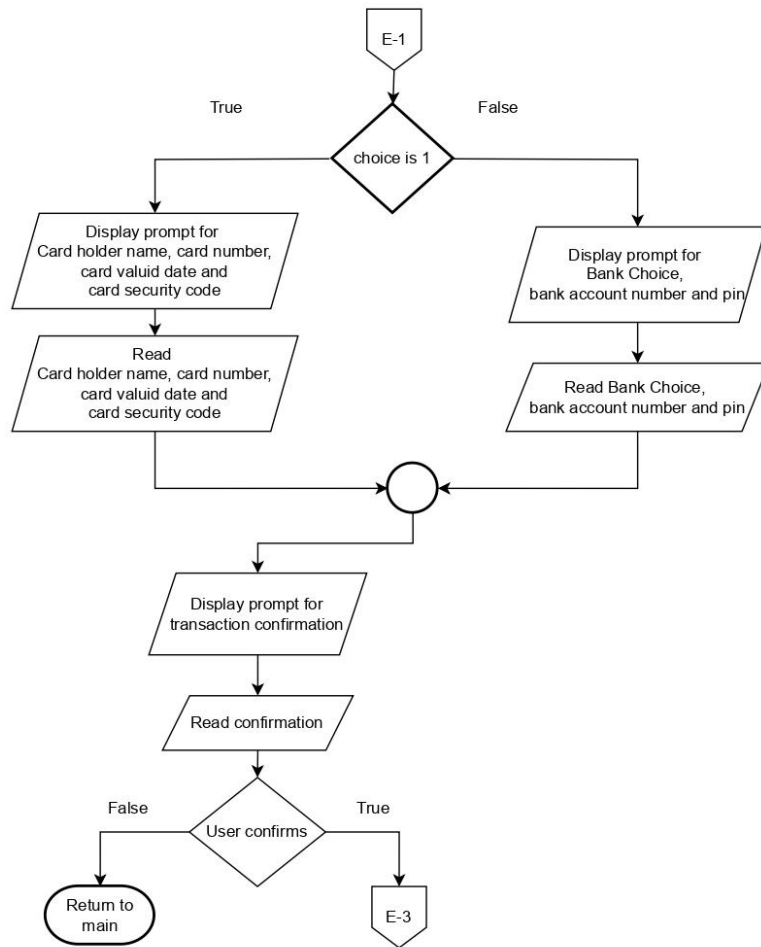


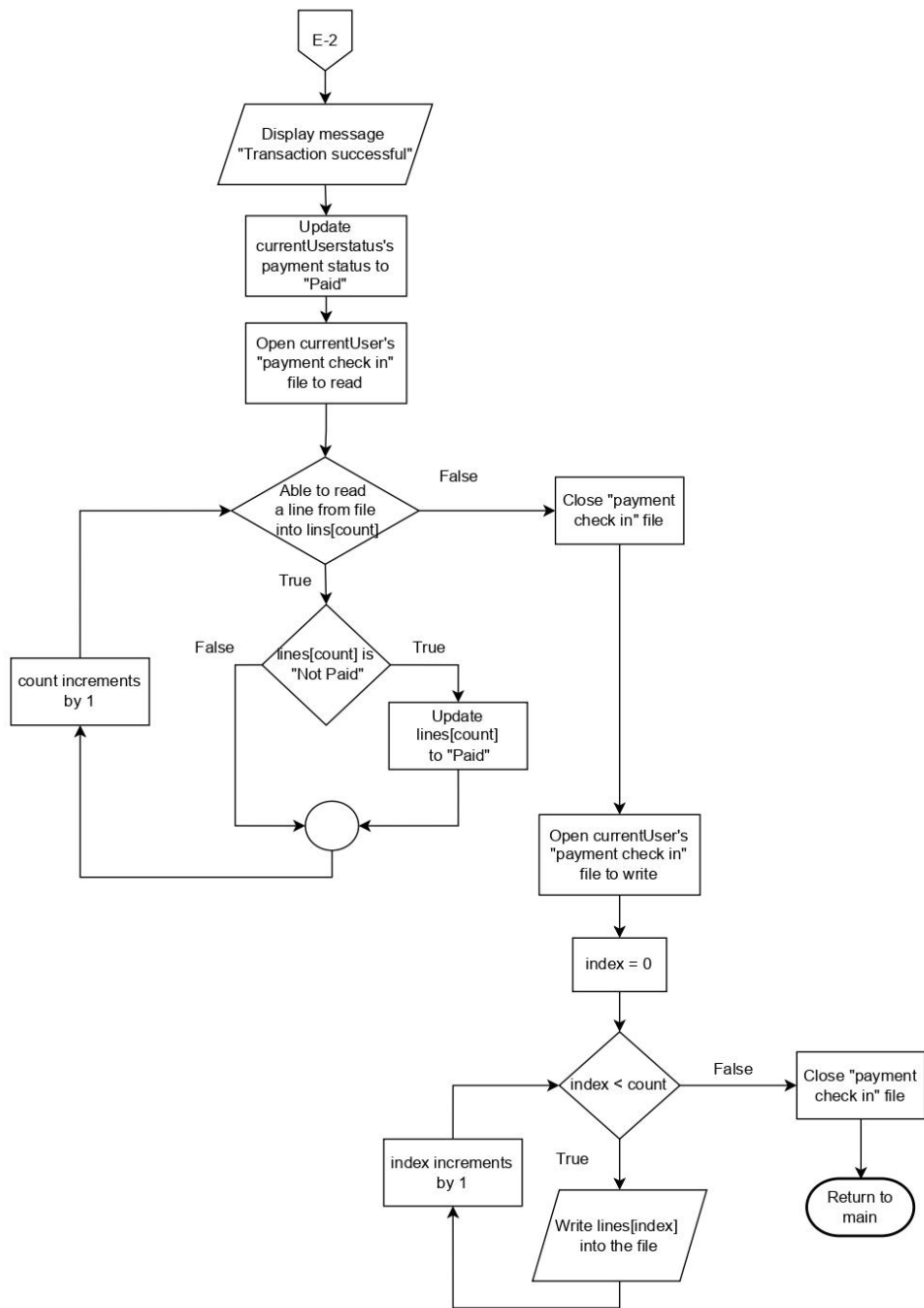


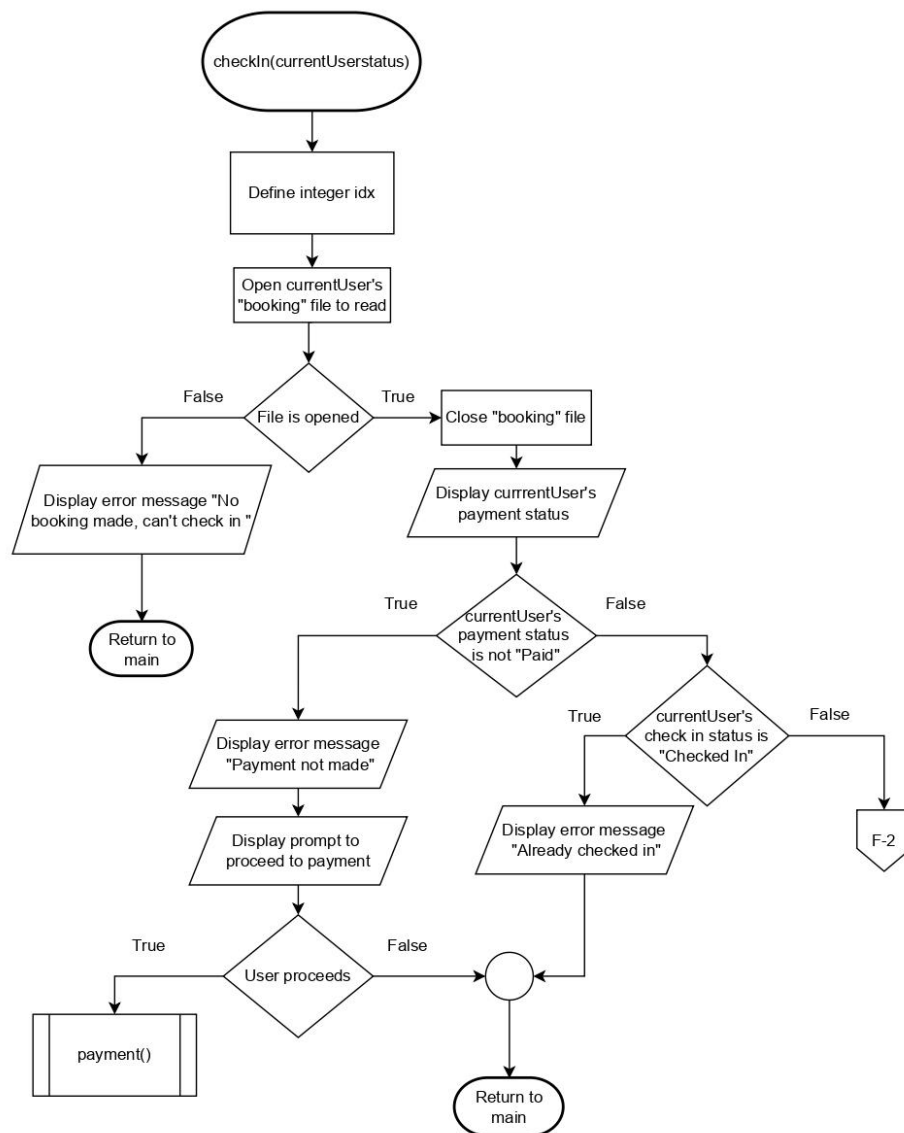


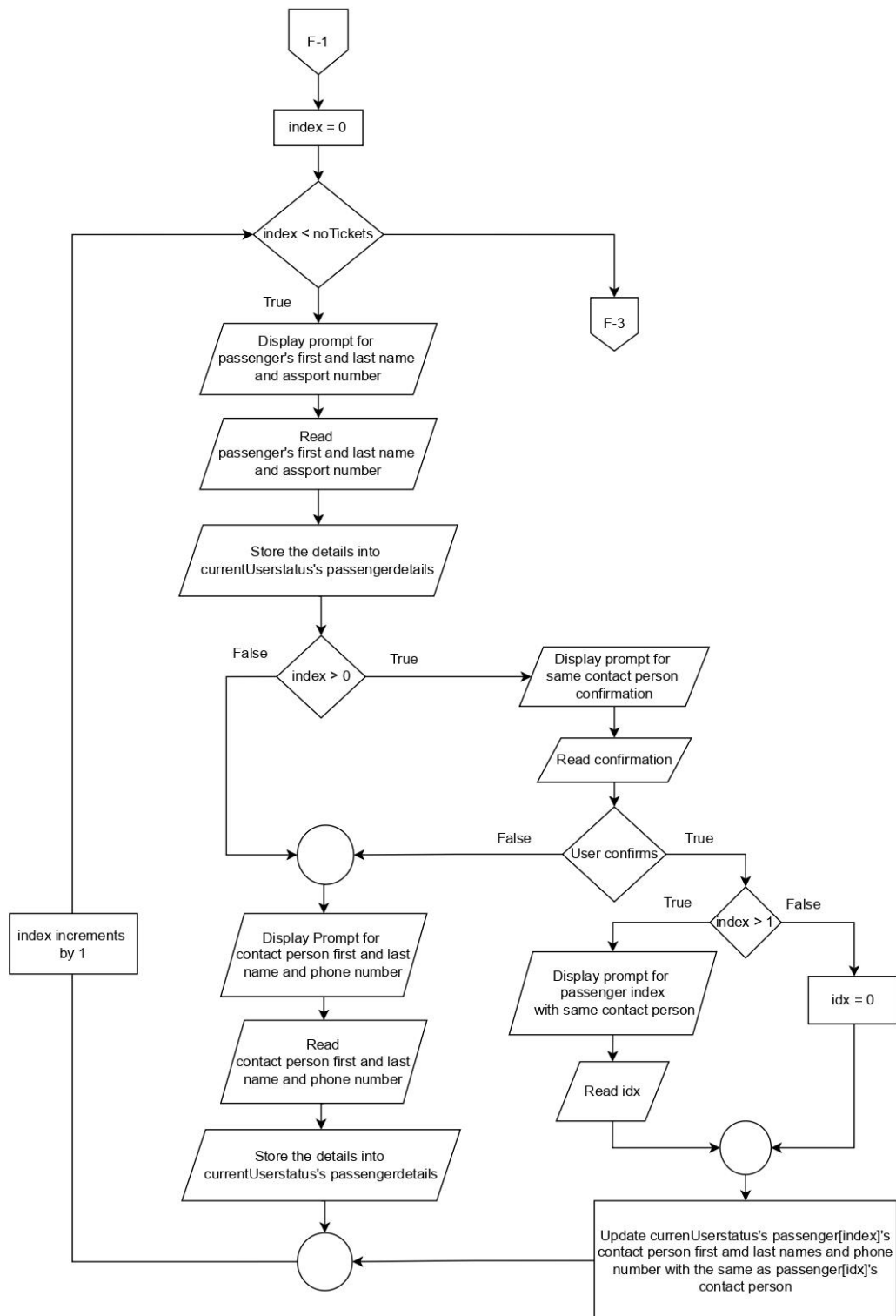


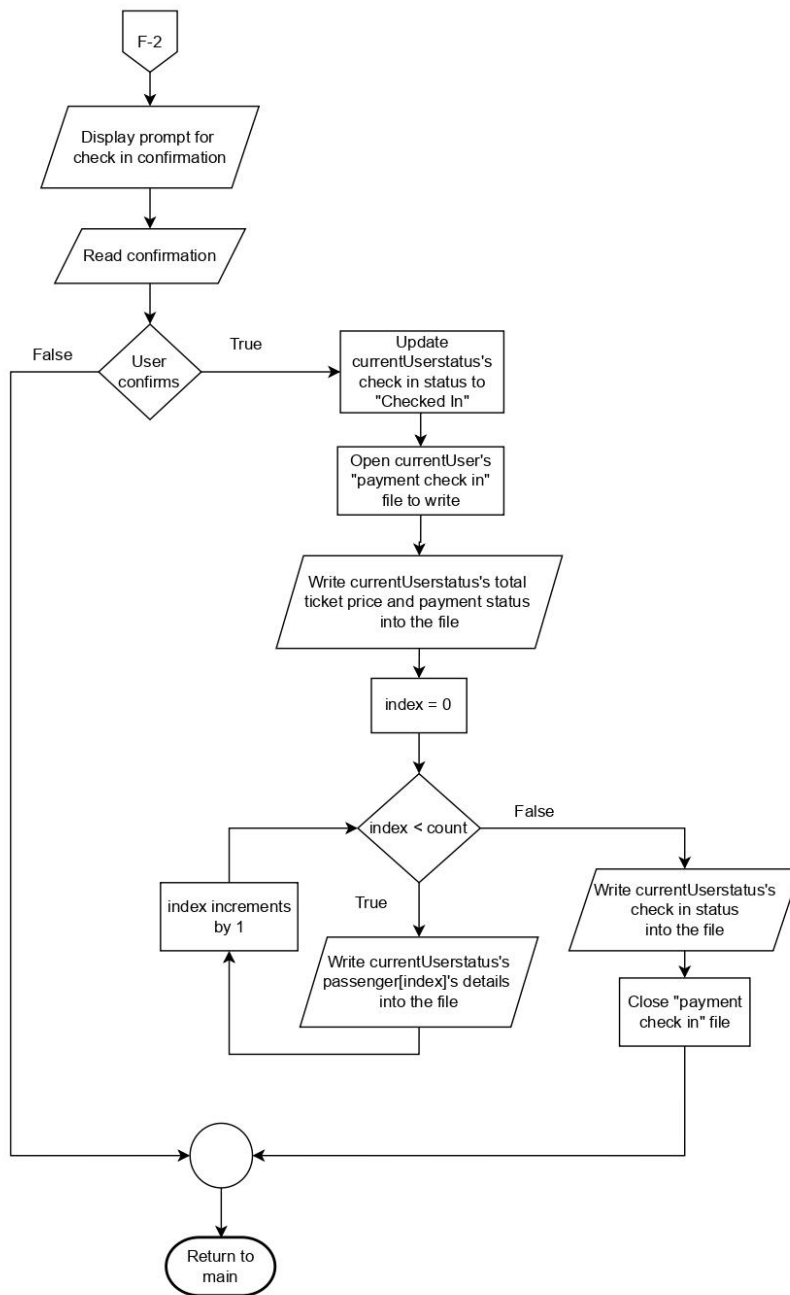


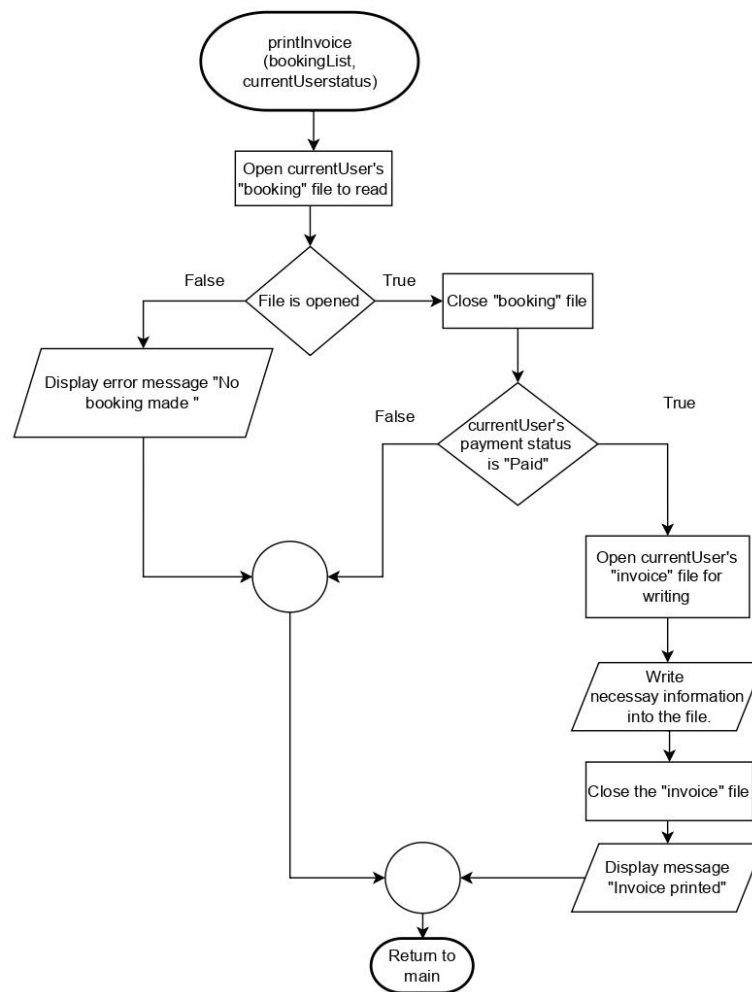












```

      -#
      +##
      +###
      .####@
    #   .#   #####+  -+#####-
    -#+ +###+ .-#####++
    #####++
    ..      +#####-
          +#       -#####@
        +#.         +###
      ++           ##
    .##.           .##
    #####-
    -+#####+.
    .+#####+.
    .--+#####+-
    --+#####+-
    --+#####+-

```

JSJK AIRLINES

★

WELCOME TO JSJK AIRLINES FLIGHT BOOKING SYSTEM

1. Register New User
2. Login
3. Quit

Choice: 1

Please key in details for registration:

First name: Jack

Last name: 2
Invalid name. Only letters and spaces allowed.

Last name: Liew

Mobile No: 0123abc
Mobile number must only be numerical with at least 10 digits!

Mobile No: 0123456789

Email: Jack.
Invalid email format.

Email: Jack@mail.
Invalid email format.

Email: Jack@gmail.com

Username: 123jack
Invalid username! It must start with a letter and 3-20 characters long.

Username: Jack

Password (At least 8 characters with 1 symbol, 1 uppercase letter, 1 number): Abc12345
Password does not meet security requirements!

Password (At least 8 characters with 1 symbol, 1 uppercase letter, 1 number): Abc12345@

Confirm to register? (y=yes, n=no): m
Please follow the instructions.

Confirm to register? (y=yes, n=no): 2
Invalid input, please enter again

Please follow the instructions.

Confirm to register? (y=yes, n=no): Y

Registration successful!

Do you want to continue login? (1=yes, 2=no): 3
Invalid input. Please enter 1 (yes) or 2 (no): |

48

Flights Available		
No	Route	Price
1	KL - Penang	RM200
2	Penang - KL	RM200
3	KL - Johor	RM200
4	Johor - KL	RM200
5	KL - Singapore	RM250
6	Singapore - KL	RM250
7	KL - Bangkok	RM300
8	Bangkok - KL	RM300

Available Departure/Return Time Slots

1.	8:00 A.M.
2.	13:00 P.M.
3.	18:00 P.M.
4.	23:00 P.M.

Number of passenger(s): 1

Please provide the details as below:

Passenger 1

First Name: Jack

Last Name: liew

Departure Flight: a

Invalid input. Please enter number between 1-8

Departure Flight: 1

Date of Departure (DD/MM/YYYY): 09/05/2025

Slot of Departure: 1

Return Flight: 2

Date of Return (DD/MM/YYYY): 08/05/2025

Invalid input. Return date must be the same as or after the departure date.

Date of Return (DD/MM/YYYY): 10/05/20254

Invalid Input. Please enter the date with format (DD/MM/YYYY).

Date of Return (DD/MM/YYYY): 10/05/2025

Slot of Return: 1

Loading...

Your booking is successful! We already save it in a file!

```
=====
                        Flights Available
=====
| No | Route                      | Price |
=====
| 1  | KL - Penang                | RM200 |
| 2  | Penang - KL                | RM200 |
| 3  | KL - Johor                 | RM200 |
| 4  | Johor - KL                 | RM200 |
| 5  | KL - Singapore             | RM250 |
| 6  | Singapore - KL             | RM250 |
| 7  | KL - Bangkok               | RM300 |
| 8  | Bangkok - KL               | RM300 |
=====

Available Departure/Return Time Slots
=====
|1. | 8:00 A.M. |
|2. | 13:00 P.M. |
|3. | 18:00 P.M. |
|4. | 23:00 P.M. |
=====

Number of passenger(s): 1

Please provide the details as below:
Passenger 1
First Name: Jack
Last Name: Liew
Departure Flight: 1
Date of Departure (DD/MM/YYYY): 10/05/2025
Slot of Departure: 1
Return Flight: 1
Date of Return (DD/MM/YYYY): /05/2025
Invalid Input. Please enter the date with format (DD/MM/YYYY).

Date of Return (DD/MM/YYYY): 11/05/2025
Slot of Return: 1

Loading...
Your booking is successful! We already save it in a file!
```

No	Route	Price
1	KL - Penang	RM200
2	Penang - KL	RM200
3	KL - Johor	RM200
4	Johor - KL	RM200
5	KL - Singapore	RM250
6	Singapore - KL	RM250
7	KL - Bangkok	RM300
8	Bangkok - KL	RM300

Available Departure/Return Time Slots

1.	8:00 A.M.
2.	13:00 P.M.
3.	18:00 P.M.
4.	23:00 P.M.

Number of passenger(s): 3

Please provide the details as below:

Passenger 1

First Name: John

Last Name: Liew

Departure Flight: 1

Date of Departure (DD/MM/YYYY): 10/05/2025

Slot of Departure: 1

Return Flight: 11/05/2025

Invalid input. Please enter number between 1-8

Return Flight: 2

Date of Return (DD/MM/YYYY): 11/05/2025

Slot of Return: 1

Passenger 2

First Name: Jack

Last Name: Liew

Departure Flight: 1

Date of Departure (DD/MM/YYYY): 10/05/2025

Slot of Departure: 1

Return Flight: 1

The input must same with the previous passenger.

Return Flight: 2

Date of Return (DD/MM/YYYY): 11/05/2025

Slot of Return: 1

Passenger 3

First Name: James

Last Name: Liew

Departure Flight: 1

Date of Departure (DD/MM/YYYY): 10/05/2025

Slot of Departure: 1

Return Flight: 1

The input must same with the previous passenger.

Return Flight: 2

Date of Return (DD/MM/YYYY): 11/05/2025

Slot of Return: 1

Loading...

Your booking is successful! We already save it in a file!

```

=====
                        Flights Available
=====
| No | Route                                | Price |
=====
| 1  | KL - Penang                         | RM200 |
| 2  | Penang - KL                         | RM200 |
| 3  | KL - Johor                         | RM200 |
| 4  | Johor - KL                         | RM200 |
| 5  | KL - Singapore                     | RM250 |
| 6  | Singapore - KL                     | RM250 |
| 7  | KL - Bangkok                       | RM300 |
| 8  | Bangkok - KL                       | RM300 |
=====

Available Departure/Return Time Slots
-----
| 1. | 8:00 A.M. |
| 2. | 13:00 P.M. |
| 3. | 18:00 P.M. |
| 4. | 23:00 P.M. |
-----

Change passenger name? (y=yes, n=no): n

Please select the item to amend
1. Departure Flight
2. Departure Date
3. Departure Slot
4. Return Flight
5. Return Date
6. Return Slot
Select: 6

Follow the details of the 1st Passenger Liew Jack
-----

    Old return slot: 1
    New return slot: a
Invalid input. Please enter number between 1-4

    Old return slot: 1
    New return slot: 2

Confirm Amendment? (y=yes, n=no): a
Please follow the instructions.

Confirm Amendment? (y=yes, n=no): y

Loading...
Your booking is successfully edited!

```

```
Payment Status: Not Paid
Payment Must be Made Before Check In!
Press any key to continue . . . |
```

```
Going to Payment
Press any key to continue . . .
Total Flight Ticket(s): 1
-----
Departure Flight: 09/05/2025, KL - Penang, 8:00 A.M.
Return Flight: 10/05/2025, Penang - KL, 13:00 P.M.
Total Payment: (RM 200 + RM 200) * 1 = RM 400
Continue for payment? (y=yes, n=no): y

Please choose your payment method:
1. Credit Card/Debit Card
2. Bank Transfer
Choice          :1

Transfer Ammount      : RM401
Invalid input, the amount due is RM 400
Transfer Ammount      : RM400
Card Holder Name      :Jack
Card Number           :000011223
The card number must be 16 digits.
Card Number           :1234567891023456
Card Valid Date(MM/YYYY) :4/2025
Invalid card, expired!
Card Valid Date(MM/YYYY) :6/2025
Security Code         :123
-----
Confirm Transaction? (y=yes, n=no) :y
-----
Transaction Successful!
Press any key to continue . . . |
```

```

Going to Payment
Press any key to continue . . .
Total Flight Ticket(s): 3
-----
Departure Flight: 10/05/2025, KL - Penang, 8:00 A.M.
Return Flight: 11/05/2025, Penang - KL, 8:00 A.M.
Total Payment: (RM 200 + RM 200) * 3 = RM 1200
Continue for payment? (y=yes, n=no):
Please follow the instructions.
Continue for payment? (y=yes, n=no): y

Please choose your payment method:
1. Credit Card/Debit Card
2. Bank Transfer
Choice :2

Transfer Ammount : RM1200
Bank Choice :public bank
Bank Account Number :1234123412341234
Bank Account Pin :adb
Invalid input, please enter the correct numbers only!
Bank Account Pin :123423
-----
Confirm Transaction? (y=yes, n=no) :y
-----
Transaction Successful!
Press any key to continue . . . |

```

```

Payment Status: Paid
~Proceed to Check In~
Press any key to continue . . .

Passenger 1
First Name :Jack
Last Name :Liew
Passport Number (eg.A12345):A12345
Contact Person First Name: Jack
Contact Person Last Name: liew
Invalid input, please enter again
Contact Person Last Name: Liew
Contact Person Phone Number: 01212121212
Confirm Check in? (y=yes, n=no): 3
Invalid input, please enter again
Please follow the instructions.
Confirm Check in? (y=yes, n=no): y|

```

Payment Status: Paid
~Proceed to Check In~
Press any key to continue . . .

Passenger 1
First Name :John
Last Name :Liew
Passport Number (eg.A12345):A12345
Contact Person First Name: Chris
Contact Person Last Name: Hemsworth
Contact Person Phone Number: 01234567891

Passenger 2
First Name :Jack
Last Name :Liew
Passport Number (eg.A12345):B12345
Same Contact Person as Previous? (y=yes, n=no): n
Contact Person First Name: Tom
Contact Person Last Name: Holland
Contact Person Phone Number: 016234567891
Invalid Phone Number Length!Contact Person Phone Number: 01623456789

Passenger 3
First Name :James
Last Name :Liew
Passport Number (eg.A12345):C12345
Same Contact Person as Previous? (y=yes, n=no): y

Which Same Contact Person?: 3

Invalid choice, Please Choose Between (1-2)
Which Same Contact Person?: 1
Contact Person First Name: Tom
Contact Person Last Name: Holland
Contact Person Phone Number: 01623456789

Confirm Check in? (y=yes, n=no): |

Invoice Printed, Please Check Your Folder!
Press any key to continue . . . |

Source Code

```
#include <iostream>
#include <iomanip>
#include <string>
#include <fstream>
#include <cctype>
#include <ctime> //libraries starting from here are used for decoration
#include <windows.h>
#include <conio.h>
#include <chrono>
#include <thread>
using namespace std;

struct names {
    string f_Name;
    string l_Name;
};
struct registerDetails {
    names userFullname;
    string userMobilenum;
    string userEmail;
    string username;
    string password;
};
struct bookingDetails {
    names passengerFullname;
    int depFlightnum = 0;
    string depDate = "";
    int depTime = 0;
    int retFlightnum = 0;
    string retDate = "";
    int retTime = 0;
};
struct checkIndetails {
    names passengerFullname;
    string passportNum;
    names contactPerson;
    string contactPerson_Mbnum;
};
struct passengerStatus {
    double ttlFlighttics;
    string paymentStatus;
    checkIndetails passenger[10];
    string checkinStatus;
};
#define SIZE 100
string currentUser;
int noTickets = 0;
bookingDetails bookingList[SIZE];
passengerStatus currentUserstatus;

void menu();
void FlightSchedule();
```

```

void registration();
void readUser(registerDetails[], int& count);
int login(registerDetails[], int count);
void readBooking(bookingDetails bookingList[], int& size);
void performBooking();
void editBooking(bookingDetails bookingList[], int size);
void readPaymentCheckIn(passengerStatus& currentUserstatus);
void payment();
void checkIn(passengerStatus& currentUserstatus);
void printInvoice(bookingDetails flight[], passengerStatus passenger);

bool isValidName(const string& name);
bool isValidMobile(const string& mobile);
bool isValidEmail(const string& email);
bool isValidUsername(const string& username);
bool isValidPassword(const string& password);
bool checkAlphabet(string inputString);
bool checkFlightNum(int inputInteger);
bool checkSlotNum(int inputInteger);
bool checkDate(string inputString);
bool checkValid(string inputString, string inputfordate);
bool checkSamedetailInt(int inputInteger, int firstinputInt, int count);
bool checkSamedetailStr(string inputString, string firstinputStr, int
count);
bool checkNochangeInt(int originalInteger, int inputInteger);
bool checkNochangeDate(string originalString, string inputString);
bool checkallName(string inputfirstName, string inputlastName,
bookingDetails bookingList[], int currentIndex);
bool checkSameName(string inputfirstName, string inputlastName,
bookingDetails bookingList[], int currentIndex);
bool checkselectNum(int select);
bool validateAlpha(string& input);
bool validateConfirmation(string input, char& choice);
bool validateDigit(string input);
bool checkPassengername(string FName, string Lname, bookingDetails
bookingList[], int idx);
bool checkPassport(string& passport);
void showLoading();
void loadingPlaneBar(int delayMs);
void banner();
void printThankyoubanner();
void setColor(int color) {
    SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HA
NDLE), color);
}

int main() {
    int choice, contrl, select, attempts = 0;;
    bool loggedIn = false;
    string option;
    double flightPrice[8] =
{ 200,200,200,200,250,250,300,300 }, ttlprice;
    do {
        banner();

```

```

        cout << " 1. Register New User\n 2.
Login\n 3. Quit\n Choice: ";
        cin >> choice;
        cin.ignore();
        if (choice == 1) {
            //function call registration to
            register new user
            registration();
            cout << "Do you want to continue
login? (1-yes, 2-no): ";
            while (!(cin >> contrl) || (contrl !=
1 && contrl != 2)) {
                cout << "Invalid input. Please
enter 1 (yes) or 2 (no): ";
                cin.clear();
                cin.ignore();
            }
            if (contrl == 2) {
                system("cls");
                printThankyoubanner();
                return 0;
            }
        }
        else if (choice == 2) {
            registerDetails users[SIZE];
            int userCount = 0;
            //function call readUserDetails to
            read the users' details from "user.txt"
            readUser(users, userCount);
            int loginResult = login(users,
userCount);
            if (loginResult == 1) {
                cout << "Login successful!" <<
endl;
                loggedIn = true;
                break;
            }
            else if (loginResult == 2) {
                cout << "Password incorrect." <<
endl;
                attempts++;
                if (attempts < 3) {
                    setColor(4);
                    cout << "You have " << 3 -
attempts << " more chances. " << endl;
                    setColor(7);
                }
            }
            else if (loginResult == 3) {
                cout << "Username incorrect." <<
endl;
                attempts++;
                if (attempts < 3) {
                    setColor(4);

```

```

                                cout << "You have " << 3 -
attempts << " more chances. " << endl;                                setColor(7);
                                }
                                }
                                else {
not correct.\nIf you have not registered, please register first!" << endl;    cout << "Username and password
                                }
                                if (attempts >= 3) {
attempts. Please wait 3 seconds";        setColor(6);
                                cout << "Too many failed
                                for (int i = 0; i < 3; i++) {
second                                Sleep(1000); // wait 1
                                cout << ".";
                                cout.flush(); // make sure
dot prints immediately
                                } // delay for 3 seconds
                                cout << endl;
                                attempts = 0; // reset counter
if you want to allow retry
                                setColor(7);
                                }
                                cout << "Do you want to continue
register/login? (1=yes, 2=no): ";        while (!(cin >> contrl) || (contrl !=
1 && contrl != 2)) {                                cout << "Invalid input. Please
                                cin.clear();
                                cin.ignore();
                                }
                                if (contrl == 2) {
enter 1 (yes) or 2 (no): ";        system("cls");
                                printThankyoubanner();
                                return 0;
                                }
                                }
                                else if (choice == 3) {
                                system("cls");
                                printThankyoubanner();
                                return 0;
                                }
                                else {
to enter 1, 2, or 3. " << endl;        cout << "Invalid choice. Please enter
                                cin.clear();
                                cin.ignore();
                                contrl = 1;
                                }
                                system("cls");

```

```

} while (contRL == 1);
int cont = 1;
loadingPlaneBar(200);
do {
    system("cls");
    banner();
    cout << "Username: " << currentUser << endl;
    menu();
    readBooking(bookingList, noTickets);
    readPaymentCheckIn(currentUserstatus);
    while (true) {
        cout << "Choice: ";
        getline(cin, option);
        if (!validateDigit(option)) {
            system("pause");
            system("cls");
            menu();
            continue;
        }
        select = stoi(option);
        if (!checkselectNum(select)) {
            menu();
            continue;
        }
        else {
            system("cls");
            break;
        }
    }
    if (select == 1) {
        performBooking();
    }
    else if (select == 2) {
        editBooking(bookingList, noTickets);
    }
    else if (select == 3) {
        payment();
    }
    else if (select == 4) {
        checkIn(currentUserstatus);
    }
    else if (select == 5) {
        printInvoice(bookingList,
currentUserstatus);
    }
    else if (select == 6) {
        cont = 0;
        system("cls");
        printThankyoubanner();
    }
    else
    {
        cont = 1;
        setColor(4);

```

```

        cout << "Not available" << endl;
        setColor(7);
    }
    if (select == 1 || select == 2) {
        ttlprice =
flightPrice[bookingList[0].depFlightnum - 1] +
flightPrice[bookingList[0].retFlightnum - 1] * noTickets;
        ofstream outFile(currentUser +
"_paymentCheckIn.txt");
        ofstream outFile1(currentUser +
"_invoice.txt");
        if (outFile.is_open()) {
            outFile << fixed <<
setprecision(0) << ttlprice << '\n';
            outFile << "Not Paid\n";
            for (int i = 0; i < noTickets *
6; i++) {
                outFile << "NULL\n";
            }
            outFile << "Not Checked In";
        }
        outFile.close();
        outFile1 << "NULL";
        outFile1.close();
    }
} while (cont == 1);
return 0;
}

void menu() {
    setColor(14); // Light cyan for border and title
    cout << "+-----" << endl;
    cout << "|                AIRLINE MENU SYSTEM" << endl;
    cout << "+-----" << endl;
    cout << "| 1. Book Flight Ticket(s)" << endl;
    cout << "| 2. Edit Booking" << endl;
    cout << "| 3. Perform Transaction" << endl;
    cout << "| 4. Check In Flight" << endl;
    cout << "| 5. Print Invoice" << endl;
    cout << "| 6. Quit" << endl;
    cout << "+-----" << endl;

    setColor(7); // Reset to default
}

```

```

void FlightSchedule() {
    setColor(3);
    cout << setw(50) <<
    "===== " << endl;
    cout << right << setw(38) << "Flights Available"
    << endl;
    cout << setw(50) <<
    "===== " << endl;
    cout << left << "|" << setw(3) << "No" << "|" <<
    setw(40) << "Route"
    << "|" << setw(10) << "Price" << " |" <<
    endl;
    cout << "-----"
    "-----" << endl;
    cout << "|" << setw(3) << "1" << "|" << setw(40)
    << "KL - Penang"
    << "|" << RM << setw(8) << "200" << " |" <<
    endl;
    cout << "|" << setw(3) << "2" << "|" << setw(40)
    << "Penang - KL"
    << "|" << RM << setw(8) << "200" << " |" <<
    endl;
    cout << "|" << setw(3) << "3" << "|" << setw(40)
    << "KL - Johor"
    << "|" << RM << setw(8) << "200" << " |" <<
    endl;
    cout << "|" << setw(3) << "4" << "|" << setw(40)
    << "Johor - KL"
    << "|" << RM << setw(8) << "200" << " |" <<
    endl;
    cout << "|" << setw(3) << "5" << "|" << setw(40)
    << "KL - Singapore"
    << "|" << RM << setw(8) << "250" << " |" <<
    endl;
    cout << "|" << setw(3) << "6" << "|" << setw(40)
    << "Singapore - KL"
    << "|" << RM << setw(8) << "250" << " |" <<
    endl;
    cout << "|" << setw(3) << "7" << "|" << setw(40)
    << "KL - Bangkok"
    << "|" << RM << setw(8) << "300" << " |" <<
    endl;
    cout << "|" << setw(3) << "8" << "|" << setw(40)
    << "Bangkok - KL"
    << "|" << RM << setw(8) << "300" << " |" <<
    endl;
    cout << "-----"
    "-----" << endl << endl;
    cout << setw(45) << "Available Departure/Return
    Time Slots" << endl;
    cout << "-----"
    << endl;
    cout << left;
}

```

```

        cout << "|" << "1. " << setw(5) << "|" << setw(29)
<< "8:00 A.M." << "|" << endl;
        cout << "|" << "2. " << setw(5) << "|" << setw(29)
<< "13:00 P.M." << "|" << endl;
        cout << "|" << "3. " << setw(5) << "|" << setw(29)
<< "18:00 P.M." << "|" << endl;
        cout << "|" << "4. " << setw(5) << "|" << setw(29)
<< "23:00 P.M." << "|" << endl;
        cout << "-----"
<< endl;

        setColor(7);
}

// Function registration --> To register new user and save into "user.txt"
void registration() {
    registerDetails newUser;
    registerDetails users[SIZE];
    int userCount = 0;

    readUser(users, userCount);
    bool exists = false;
    cout << "Please key in details for registration:"

<< endl;

    // First Name
    do {
        cout << "First name: ";
        getline(cin, newUser.userFullname.f_Name);
        if (newUser.userFullname.f_Name.empty()) {
            setColor(4);
            cout << "First name cannot be empty!"

<< endl;

            setColor(7);
        }
        //call from the validation name function to
        check the format
        else if
        (!isValidName(newUser.userFullname.f_Name)) {
            setColor(4);
            cout << "Name can only contain letters
and spaces!" << endl;

            setColor(7);
        }
    } while (newUser.userFullname.f_Name.empty()
|| !isValidName(newUser.userFullname.f_Name));

    // Last Name
    do {
        cout << "Last name: ";
        getline(cin, newUser.userFullname.l_Name);
        if (newUser.userFullname.l_Name.empty()) {
            setColor(4);
            cout << "Last name cannot be empty!"

<< endl;

```



```

        setColor(7);
    }
    else if
(!isValidName(newUser.userFullname.l_Name)) {
        setColor(4);
        cout << "Invalid name. Only letters
and spaces allowed." << endl;
        setColor(7);
    }
    } while (newUser.userFullname.l_Name.empty()
|| !isValidName(newUser.userFullname.l_Name));

    // Mobile number
    do {
        cout << "Mobile No: ";
        getline(cin, newUser.userMobilenum);
        if (newUser.userMobilenum.empty())
        {
            setColor(4);
            cout << "Mobile number cannot be
empty!" << endl;
            setColor(7);
        }
        //call from the validation mobilenum
function to check the format
        else if
(!isValidMobile(newUser.userMobilenum)) {
            setColor(4);
            cout << "Mobile number must only be
numerical with at least 10 digits!" << endl;
            setColor(7);
        }
    } while (newUser.userMobilenum.empty()
|| !isValidMobile(newUser.userMobilenum));

    // Email
    do {
        cout << "Email: ";
        getline(cin, newUser.userEmail);
        if (newUser.userEmail.empty()) {
            setColor(4);
            cout << "Email cannot be empty!" <<
endl;
            setColor(7);
        }
        //call from the validation email function to
check the format
        else if (!isValidEmail(newUser.userEmail)) {
            setColor(4);
            cout << "Invalid email format." <<
endl;
            setColor(7);
        }
    }

```

```

        } while (newUser.userEmail.empty()
|| !isValidEmail(newUser.userEmail));

        // Username
        do {
            //assume no same username before in the
record
            cout << "Username: ";
            getline(cin, newUser.username);

            if (newUser.username.empty()) {
                setColor(4);
                cout << "Username cannot be empty!" <<
endl;

                setColor(7);
                continue;
            }

            //check if the username exists in the
current records
            for (int i = 0; i < userCount; ++i) {
                if (users[i].username ==
newUser.username) {
                    exists = true;
                    break;
                }
            }
            if (exists) {
                setColor(4);
                cout << "Username already exists!" <<
endl;

                setColor(7);
            }

            // call from the validation username
function to check the format
            else if (!isValidUsername(newUser.username))
            {
                setColor(4);
                cout << "Invalid username! It must
start with a letter and 3-20 characters long." << endl;
                setColor(7);
            }

            // repeat the loop if the username is empty,
invalid or already taken
        } while (newUser.username.empty()
|| !isValidUsername(newUser.username) || exists);

        // Password
        do {
            cout << "Password (At least 8 characters
with 1 symbol, 1 uppercase letter, 1 number): ";
            getline(cin, newUser.password);

```

```

        if (newUser.password.empty()) {
            setColor(4);
            cout << "Password cannot be empty!" <<
endl;
            setColor(7);
        }

        // call from the validation password
function to check the format
        else if (!isValidPassword(newUser.password))
{
            setColor(4);
            cout << "Password does not meet
security requirements!" << endl;
            setColor(7);
        }

        } while (newUser.password.empty()
|| !isValidPassword(newUser.password));

        //Confirm registration
        char confirm;
        string input;

        while (true) {
            cout << "Confirm to register? (y=yes, n=no):
";
            getline(cin, input);

            if (!validateConfirmation(input, confirm))
                continue;
            else
                break;
        }

        if (confirm == 'n') {
            cout << "Registration cancelled." << endl;
            return;
        }

        // Save to file
        ofstream outfile("user.txt", ios::app);
        if (outfile) {
            outfile << newUser.userFullname.f_Name <<
"\n"
            << newUser.userFullname.l_Name << "\n"
            << newUser.userMobilenum << "\n"
            << newUser.userEmail << "\n"
            << newUser.username << "\n"
            << newUser.password << endl;
            outfile.close();

            setColor(2);

```

```

        cout << "Registration successful!\n";
        setColor(7);
    }
}

// Function readUser --> Read (ifstream) all the registered users listed
in "user.txt"
// and store the details into the respective parameters

void readUser(registerDetails users[], int& count) {
    ifstream infile("user.txt");
    //initialize the user count to 0
    count = 0;

    //read user data until end of file or array
    reaches its max size
    while (!infile.eof() && count < SIZE) {
        registerDetails userRecord;
        getline(infile,
userRecord.userFullname.f_Name);
        getline(infile,
userRecord.userFullname.l_Name);
        getline(infile, userRecord.userMobilenumber);
        getline(infile, userRecord.userEmail);
        getline(infile, userRecord.username);
        getline(infile, userRecord.password);

        if (infile.fail())
            break;

        users[count++] = userRecord;
    }

    infile.close();
}

// Function login --> For user to login with the registered username and
password
int login(registerDetails users[], int count) {
    string inputUsername, inputPassword;
    char ch;
    //assume no initial username and password found
    bool usernameFound = false;
    bool passwordFound = false;

    cout << "Enter username: ";
    getline(cin, inputUsername);

    cout << "Enter password: ";
    while (true) {
        ch = _getch(); // Get character without echo
        if (ch == 13) break; // Enter key
        else if (ch == 8) { // Backspace
            if (!inputPassword.empty()) {

```

```

        cout << "\b \b";
        inputPassword.pop_back();
    }
}
else if (isprint(ch)) {
    inputPassword += ch;
    cout << '*';
}
}
cout << endl;

//check if either input is empty
if (inputUsername.empty() ||
inputPassword.empty()) {
    setColor(4);
    cout << "Username and password cannot be
empty!" << endl;
    setColor(7);
    return 4; //Both incorrect
}

// Use for loop to check whether the registered
username and password exist in the array
for (int i = 0; i < count; ++i) {
    if (users[i].username == inputUsername &&
users[i].password == inputPassword) {
        //Set the global curentUser to logged-
in username
        currentUser = inputUsername;
        return 1; //Both correct
    }
    else if (users[i].username == inputUsername
&& users[i].password != inputPassword) {
        usernameFound = true; //Username
correct, password incorrect
    }
    else if (users[i].username != inputUsername
&& users[i].password == inputPassword) {
        passwordFound = true; //Password
correct, username incorrect
    }
}

if (usernameFound) {
    return 2; //Username correct, password wrong
}
else if (passwordFound) {
    return 3; //Password correct, username wrong
}
else {
    return 4; //Both wrong
}
}

```

```

// Function performBooking --> For user input the booking passenger
details and save in struct array and "currentUse_Booking.txt"
void performBooking() {

    char cont;
    string input;

    bookingDetails bookingList[SIZE];
    ifstream checkFile(currentUser + "_Booking.txt");

    // Check booking file is available or not
    if (checkFile) {

        cout << "Sorry, It have found that you have
already have booking record." << endl;
        do {
            cout << "Do you want to edit your
booking?\n(Y)es or (N)o: ";

            getline(cin, input);

            // Check user input not empty input
            and only alphabet
            } while (!validateConfirmation(input,
cont));

            if (cont == 'y') {
                system("cls");
                editBooking(bookingList, noTickets);
            }

            else
                return;

            checkFile.close();
        }

        ofstream bookFile(currentUser + "_Booking.txt");
        FlightSchedule();

        do {
            // Enter number of Passenger
            cout << "Number of passenger(s): ";
            cin >> noTickets;
            cin.ignore();

            // Check user input is digit only or not
            if (cin.fail()) {
                cin.clear();
                cin.ignore(100, '\n');
                setColor(4);
                cout << "Invalid input! Please enter a
valid number" << endl;

                setColor(7);
            }
        }
    }
}

```

```

1          // Check the number of ticket must more than
          else if (noTickets < 1) {
              setColor(4);
              cout << "Please enter a valid number
of passengers (1 or more). " << endl;
              setColor(7);
          }

          } while (cin.fail() || noTickets < 1);

          cout << "\nPlease provide the details as below: "
<< endl;

          for (int count = 0; count < noTickets; count++)
          {
              setColor(9);
              cout << "Passenger " << count + 1 << endl;
              setColor(7);

              do {
                  // Let user enter she/he first name
                  do {
                      cout << "First Name: ";
                      getline(cin,
bookingList[count].passengerFullname.f_Name);

                      //check the input is alphabet or
not and is it empty
                      } while
(!validateAlpha(bookingList[count].passengerFullname.f_Name));

                  // Let user enter she/he last name
                  do {
                      cout << "Last Name: ";
                      getline(cin,
bookingList[count].passengerFullname.l_Name);

                      //check the input is alphabet or
not and is it empty
                      } while
(!validateAlpha(bookingList[count].passengerFullname.l_Name));

                  // Check the input is same with
previous passenger or not
                  } while
(!checkSameName(bookingList[count].passengerFullname.f_Name,
bookingList[count].passengerFullname.l_Name, bookingList, count));

                  //Save in file
                  bookFile <<
bookingList[count].passengerFullname.f_Name << endl;

```

```

        bookFile <<
bookingList[count].passengerFullname.l_Name << endl;

        //Enter Departure flight number
        do {
            cout << "Departure Flight: ";
            cin >>
bookingList[count].depFlightnum;

            //check the input is same or not and
the format of the input (must enter number between 1-8 only)
        } while
(!checkFlightNum(bookingList[count].depFlightnum)
|| !checkSamedetailInt(bookingList[count].depFlightnum,
bookingList[0].depFlightnum, count));
        bookFile << bookingList[count].depFlightnum
<< endl;

        //Enter date of departure
        do {
            cout << "Date of Departure
(DD/MM/YYYY): ";

            cin >> bookingList[count].depDate;

            //check the input is same or not and
the format of the date
        } while
(!checkDate(bookingList[count].depDate)
|| !checkSamedetailStr(bookingList[count].depDate, bookingList[0].depDate,
count));
        bookFile << bookingList[count].depDate <<
endl;

        //Enter slot of departure
        do {
            cout << "Slot of Departure: ";
            cin >> bookingList[count].depTime;

            //check the input is same or not and
the format of the input (must enter number between 1-4 only)
        } while
(!checkSlotNum(bookingList[count].depTime)
|| !checkSamedetailInt(bookingList[count].depTime, bookingList[0].depTime,
count));
        bookFile << bookingList[count].depTime <<
endl;

        //Enter return flight number
        do {
            cout << "Return Flight: ";
            cin >>
bookingList[count].retFlightnum;

```



```

        //check the input is same or not and
the format of the input (must enter number between 1-8 only)
        } while
(!checkFlightNum(bookingList[count].retFlightnum)
|| !checkSamedetailInt(bookingList[count].retFlightnum,
bookingList[0].retFlightnum, count));
        bookFile << bookingList[count].retFlightnum
<< endl;

        //Enter date of return
do {
        cout << "Date of Return (DD/MM/YYYY):
";

        cin >> bookingList[count].retDate;

        //check the input is same or not and
the format of the date and must late than date of departure flight
        } while
(!checkDate(bookingList[count].retDate)
|| !checkValid(bookingList[count].retDate, bookingList[count].depDate)
|| !checkSamedetailStr(bookingList[count].retDate, bookingList[0].retDate,
count));
        bookFile << bookingList[count].retDate <<
endl;

        //Enter slot of return
do {
        cout << "Slot of Return: ";
        cin >> bookingList[count].retTime;

        //check the input is same or not and
the format of the input (must enter number between 1-4 only)
        } while
(!checkSlotNum(bookingList[count].retTime)
|| !checkSamedetailInt(bookingList[count].retTime, bookingList[0].retTime,
count));
        bookFile << bookingList[count].retTime <<
endl;

        cout << endl;
        cin.ignore();
    }
    bookFile.close();
    showLoading();

    setColor(2);
    cout << "Your booking is successful! We already
save it in a file!" << endl;
    setColor(7);
    cin.ignore();
}

// Function readBooking --> Read out the file and store in the array
struct and update the number of tickets to the global variable (noTickets)

```

```

void readBooking(bookingDetails bookingList[], int& noTickets) {
    noTickets = 0;
    ifstream readOutfile(currentUser +
"_Booking.txt");

    if (readOutfile.is_open()) {
        while (!readOutfile.eof())
        {
            getline(readOutfile,
bookingList[noTickets].passengerFullname.f_Name);
            getline(readOutfile,
bookingList[noTickets].passengerFullname.l_Name);
            readOutfile >>
bookingList[noTickets].depFlightnum;
            readOutfile >>
bookingList[noTickets].depDate;
            readOutfile >>
bookingList[noTickets].depTime;
            readOutfile >>
bookingList[noTickets].retFlightnum;
            readOutfile >>
bookingList[noTickets].retDate;
            readOutfile >>
bookingList[noTickets].retTime;

            if (readOutfile.fail())
                // If there are no more lines to
read from the file, break out of the loop
                break;

            readOutfile.ignore();
            ++noTickets;
        }
        readOutfile.close();
    }

    else
    {
        return; // No file
    }
}

// Function editBooking --> For user to edit passenger details, update and
save in struct array and "currentUser_Booking.txt"
void editBooking(bookingDetails bookingList[], int noTickets)
{
    bool found = false;
    char choice, confirm;
    string oldfirstName, oldlastName, newfirstName,
newlastName, newDepdate, newRetdate, input;
    int select, newDepflight, newDepTime,
newRetflight, newRetTime;
    ifstream readOutfile(currentUser +
"_Booking.txt");

```

```

        // Check booking is available or not
        if (!readOutfile) {
            cout << "No booking available, please book
your flight ticket(s)!" << endl;
            cout << "Press enter to continue . . ." <<
endl;

            cin.ignore();
            return;
        }
        readOutfile.close();

        FlightSchedule();

        do {
            // Choose to change name or not
            cout << "Change passenger name? (y=yes, n-
no): ";

            getline(cin, input);

            // Check the input is alphabet or not and is
empty or not
        } while (!validateConfirmation(input, choice));

        if (choice == 'y') {
            do {
                cout << endl;
                cout << "Which name do you want to
change? " << endl;

                //Enter original firstname and
lastname to find the name
                do {
                    cout << "Please enter the
original name to make the change (first name): ";
                    getline(cin, oldfirstName);

                    //check whether input is
alphabet and no empty
                } while
(!validateAlpha(oldfirstName));

                do {
                    cout << "Please enter the
original name to make the change (last name): ";
                    getline(cin, oldlastName);

                    //check whether input is
alphabet and no empty
                } while (!validateAlpha(oldlastName));

                for (int i = 0; i < noTickets; i++) {
                    if (oldfirstName ==
bookingList[i].passengerFullname.f_Name &&

```

```

bookingList[i].passengerFullname.l_Name)
{
    found! " << endl;

    first name
    enter the new first name: ";
    newfirstName);

    input is alphabet and no empty
    (!validateAlpha(newfirstName));

    last name
    enter the new last name: ";
    newlastName);

    input is alphabet and no empty
    (!validateAlpha(newlastName));

    // Check if the name
    is repeated with the previous original full name and others passenger name
    } while
    (!checkallName(newfirstName, newlastName, bookingList, i));

    do {
        cout << endl;
        cout << "Confirm

        getline(cin, input);
        cin.ignore();

        //Check whether have

    } while
    (!validateConfirmation(input, confirm));

    oldlastName ==
    found = true;
    cout << endl;
    setColor(2);
    cout << "The name is

    setColor(7);

    do {
        do {
            //Enter the new

            cout << "Please

            getline(cin,

            //check whether

        } while

        do {
            //Enter the new

            cout << "Please

            getline(cin,

            //check whether

        } while

```

```

                                if (confirm == 'y') {
                                    //Save in the struct
array
                                bookingList[i].passengerFullname.f_Name =
newfirstName;
                                bookingList[i].passengerFullname.l_Name =
newlastName;
                                }
                                break;
                            }
                        }

                        // If the name was not found
                        if (!found) {
                            setColor(4);
                            cout << "The name you entered
was not found. Please try again" << endl;
                            setColor(7);
                            cout << endl;
                            cin.ignore();
                        }

                        //when the name is not found and do
not confirm the amendment
                    } while (!found || confirm == 'n');
                }

                else if (choice == 'n') {
                    do {
                        cout << endl;
                        cout << "Please select the item to
amend" << endl;

                        cout << "1. Departure Flight" << endl;
                        cout << "2. Departure Date" << endl;
                        cout << "3. Departure Slot" << endl;
                        cout << "4. Return Flight" << endl;
                        cout << "5. Return Date" << endl;
                        cout << "6. Return Slot" << endl;

                        cout << "Select: ";
                        cin >> select;
                        cin.ignore();
                        cout << endl;

                        cout << "Follow the details of the 1st
Passenger " << bookingList[0].passengerFullname.l_Name << " " <<
bookingList[0].passengerFullname.f_Name << endl;
                        cout << "-----
----- " << endl;
                        if (select == 1)
                        {
                            do {

```

```

do
{
    // Input new
    departure flight
    cout << endl;
    cout << "    Old
departure flight: " << bookingList[0].depFlightnum << endl;
    cout << "    New
departure flight: ";

    cin >> newDepflight;
    cin.ignore();

    //Check the new input
    is repeated or not and format of the new input (must input number between
    1-8)
    } while
    (!checkFlightNum(newDepflight)
    || !checkNochangeInt(bookingList[0].depFlightnum, newDepflight));

    do {
        cout << endl;
        cout << "Confirm
Amendment? (y=yes, n=no): ";

        getline(cin, input);

        //Check whether have
        correct input
    } while
    (!validateConfirmation(input, confirm));

    } while (confirm == 'n');

    if (confirm == 'y')
        bookingList[0].depFlightnum
= newDepflight;
    }

    else if (select == 2) {
        do {
            do
            {
                // Enter new
                departure date
                cout << endl;
                cout << "    Old
departure date: " << bookingList[0].depDate << endl;
                cout << "    New
departure date: ";

                cin >> newDepdate;
                cin.ignore();

```

```

//Check the new input
is repeated or not and format of the new input and the date cannot late
than date of return flight
    } while
(!checkDate(newDepdate) || !checkNochangeDate(bookingList[0].depDate,
newDepdate) || !checkValid(bookingList[0].retDate, newDepdate));

do {
    cout << endl;
    cout << "Confirm

Amendment? (y=yes, n=no): ";

    getline(cin, input);

    //Check whether have
correct input
    } while
(!validateConfirmation(input, confirm));
} while (confirm == 'n');

if (confirm == 'y')
    bookingList[0].depDate =
newDepdate;
}

else if (select == 3) {
    do {
        do
        {
            //Enter new departure
slot
            cout << endl;
            cout << "  Old
departure slot: " << bookingList[0].depTime << endl;
            cout << "  New
departure slot: ";

            cin >> newDepTime;
            cin.ignore();

            //Check the new input
is repeated or not and format of the new input (must input number between
1-4)
        } while
(!checkSlotNum(newDepTime) || !checkNochangeInt(bookingList[0].depTime,
newDepTime));

        do {
            cout << endl;
            cout << "Confirm

Amendment? (y=yes, n=no): ";

            getline(cin, input);

            //Check whether have
correct input

```

```

        } while
(!validateConfirmation(input, confirm));
    } while (confirm == 'n');
    if (confirm == 'y')
        bookingList[0].depTime =
newDepTime;
    }
    else if (select == 4) {
        do {
            do {
                // Enter new return
                cout << endl;
                cout << "    Old return
flight: " << bookingList[0].retFlightnum << endl << "    New return flight:
";
                cin >> newRetflight;
                cin.ignore();

                //Check the new input
is repeated or not and format of the new input (must input number between
1-8)
            } while
(!checkFlightNum(newRetflight)
|| !checkNochangeInt(bookingList[0].retFlightnum, newRetflight));
            do {
                cout << endl;
                cout << "Confirm
Amendment? (y=yes, n=no): ";
                getline(cin, input);

                //Check whether have
correct input
            } while
(!validateConfirmation(input, confirm));
        } while (confirm == 'n');
        if (confirm == 'y')
            bookingList[0].retFlightnum
= newRetflight;
    }
    else if (select == 5) {
        do {
            do {
                // Enter new return
                date
                cout << endl;
                cout << "    Old return
date: " << bookingList[0].retDate << endl;

```



```

date: ";

cout << " New return

cin >> newRetdate;
cin.ignore();

//Check the new input
is repeated or not and format of the new input and the date cannot early
than date of departure flight
} while
(!checkDate(newRetdate) || !checkNochangeDate(bookingList[0].retDate,
newRetdate) || !checkValid(newRetdate, bookingList[0].depDate));

do {
    cout << endl;
    cout << "Confirm

Amendment? (y=yes, n=no): ";

    getline(cin, input);

    //Check whether have
correct input
} while
(!validateConfirmation(input, confirm));
} while (confirm == 'n');

if (confirm == 'y')
    bookingList[0].retDate =
newRetdate;
}

else if (select == 6) {
    do {
        do
        {
            // Enter new return

            cout << endl;
            cout << " Old return

            cout << " New return

            cin >> newRetTime;
            cin.ignore();

            //Check the new input
is repeated or not and format of the new input (must input number between
1-4)
        } while
        (!checkSlotNum(newRetTime) || !checkNochangeInt(bookingList[0].retTime,
newRetTime));

        do {
            cout << endl;
            cout << "Confirm

Amendment? (y=yes, n=no): ";

```

```

        getline(cin, input);

        //Check whether have
correct input
        } while
(!validateConfirmation(input, confirm));
        } while (confirm == 'n');
        if (confirm == 'y')
            bookingList[0].retTime =
newRetTime;
        }

        //Check whether have correct input
(input must between number 1-6)
        } while (!checkselectNum(select));
    }

    cout << endl;
    showLoading();
    setColor(2);
    cout << "Your booking is successfully edited! " <<
endl;
    setColor(7);

    //Update the information that save in struct array
to the file
    ofstream inputFile(currentUser + "_Booking.txt");
    for (int count = 0; count < noTickets; count++) {
        inputFile <<
bookingList[count].passengerFullname.f_Name << endl;
        inputFile <<
bookingList[count].passengerFullname.l_Name << endl;
        inputFile << bookingList[0].depFlightnum <<
endl;
        inputFile << bookingList[0].depDate << endl;
        inputFile << bookingList[0].depTime << endl;
        inputFile << bookingList[0].retFlightnum <<
endl;
        inputFile << bookingList[0].retDate << endl;
        inputFile << bookingList[0].retTime << endl;
    }
    inputFile.close();

    cin.ignore();
}

void readPaymentCheckIn(passengerStatus& currentUserstatus) {
    ifstream inFile;
    inFile.open(currentUser + "_paymentCheckin.txt");
    if (inFile.is_open()) {

        while (inFile >>
currentUserstatus.ttlFlighttics) {

```

```

;
inFile.ignore();
//store into global struct.
getline(inFile,
currentUserstatus.paymentStatus);
for (int i = 0; i < noTickets; i++) {
    getline(inFile,
currentUserstatus.passenger[i].passengerFullname.f_Name);
    getline(inFile,
currentUserstatus.passenger[i].passengerFullname.l_Name);
    getline(inFile,
currentUserstatus.passenger[i].passportNum);
    getline(inFile,
currentUserstatus.passenger[i].contactPerson.f_Name);
    getline(inFile,
currentUserstatus.passenger[i].contactPerson.l_Name);
    getline(inFile,
currentUserstatus.passenger[i].contactPerson_Mbnum);
}
getline(inFile,
currentUserstatus.checkinStatus);
}
}

inFile.close();

}

void payment() {
    const int flights = 8; //size for flight route and
price list
    int method, startYrnum, cardmthlen, cardYr,
cardmth, count = 0, monthNum[12] = { 1,2,3,4,5,6,7,8,9,10,11,12 },
currentMth, currentYr;;
    double flightPrice[flights] =
{ 200,200,200,200,250,250,300,300 },
    depprice =
flightPrice[bookingList[0].depFlightnum - 1],
    retprice =
flightPrice[bookingList[0].retFlightnum - 1],
    ttlprice = (depprice + retprice) *
noTickets, payment;

    string Route[flights] = { "KL - Penang" , "Penang
- KL" , "KL - Johor" , "Johor - KL" , "KL - Singapore" , "Singapore -
KL" , "KL - Bangkok" , "Bangkok - KL" };
    string Time[4] = { "8:00 A.M.", "13:00 P.M.", "18:00
P.M.", "23:00 P.M." }, proceed, choice, pay, cardHolder, cardNum,
cardValiddate, cardSeccode,
    bankChoice, bankAccnum, bankPin,
transferinput, lines[SIZE], oldText = "Not Paid", date = __TIMESTAMP__,
//Get real time date.
    monthStr = date.substr(4, 3), yearStr =
date.substr(20, 4),

```

```

        monthName[12] =
{ "Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"
};

        char confirm;

        ifstream inFile(currentUser + "_Booking.txt");
        //start to verify if user is valid to make
payment.
        //try to open the booking file if it exists
        if (!inFile.is_open()) {
            setColor(4);
            cout << "No Booking Available, please book
your flight ticket(s)!\n";
            setColor(7);
            system("pause");
            return;
        }
        else if (currentUserstatus.paymentStatus ==
"Paid") {
            setColor(2);
            cout << "Payment already completed!\n";
            setColor(7);
            system("pause");
            return;
        }

        inFile.close();

        cout << "Total Flight Ticket(s): " << noTickets <<
endl << "-----\n";
        cout << "Departure Flight: " <<
bookingList[0].depDate << ", " << Route[bookingList[0].depFlightnum - 1]
<< ", " << Time[bookingList[0].depTime - 1] << endl
        << "Return Flight: " <<
bookingList[0].retDate << ", " << Route[bookingList[0].retFlightnum - 1]
<< ", " << Time[bookingList[0].retTime - 1] << endl;
        cout << "Total Payment: (RM " << depprice << " + "
<< "RM " << retprice << ") * " << noTickets << " = RM " << ttlprice <<
endl;

        while (true) {
            cout << "Continue for payment? (y=yes, n-
no): ";
            getline(cin, proceed);

            if (!validateConfirmation(proceed, confirm))
                continue;
            else
                break;
        }

```

```

        if (confirm == 'n') {
            cout << "Returning to menu....." << endl;
            loadingPlaneBar(200);
            return;
        }
        cout << "\nPlease choose your payment method:" <<
endl;

        while (true) {
            cout << "1. Credit Card/Debit Card" << endl
                << "2. Bank Transfer " << endl
                << left << setw(15) << "Choice" <<
":";

            getline(cin, choice);

            if (!validateDigit(choice))
                continue;
            method = stoi(choice);
            if (method < 1 || method > 2) {
                setColor(4);
                cout << "Invalid input, please enter a
number between 1 and 2!\n";
                setColor(7);
            }
            else
                break;
        }
        cout << endl;
        while (true) {
            cout << left << setw(25) << "Transfer
Ammount" << ": RM";

            getline(cin, transferinput);

            if (!validateDigit(transferinput))
                continue;
            payment = stod(transferinput);
            if (payment < ttlprice || payment > ttlprice)
            {
                setColor(4);
                cout << "Invalid input, the amount due
is RM " << ttlprice << endl;
                setColor(7);
            }
            else
                break;
        }
        if (method == 1) {

            while (true) {

```

```

Holder Name" << ":";

cout << left << setw(25) << "Card
getline(cin, cardHolder);

if (!validateAlpha(cardHolder))
    continue;
else
    break;
}
while (true) {
    cout << left << setw(25) << "Card
    getline(cin, cardNum);
    if (!validateDigit(cardNum))
        continue;
    else if (cardNum.length() != 16) {
        setColor(4);
        cout << "The card number must be
16 digits." << endl;

        setColor(7);
        continue;
    }
    break;
}
while (true) {
    cout << left << setw(25) << "Card
    getline(cin, cardValidddate);
    //if the length is 7, user followed
the format, if it's 6 , user may have decided not to follow month format
    if (cardValidddate.length() == 7 &&
cardValidddate[2] == '/') {
        //decide the length of month and
start of year in input

        startYrnum = 3;
        cardmthlen = 2;
    }
    else if (cardValidddate.length() == 6
&& cardValidddate[1] == '/') {
        //same decision for the indexes
here

        startYrnum = 2;
        cardmthlen = 1;
    }
    else {
        setColor(4);
        cout << "Invalid format!\n";
        setColor(7);
        continue;
    }
}

```

```

        if
(!validateDigit(cardValidddate.substr(startYrnum, 4))
|| !validateDigit(cardValidddate.substr(0, cardmthlen)))
            continue;

        else {
            //if the month and year are only
            numbers, check the range

            cardYr =
stoi(cardValidddate.substr(startYrnum, 4));
            cardmth =
stoi(cardValidddate.substr(0, cardmthlen));
            if ((cardYr < 0 || cardYr>2060)
|| (cardmth < 0 || cardmth>12)) {
                setColor(4);
                cout << "Invalid card valid
dates, please enter the correct date.\n";
                setColor(7);
                continue;
            }
            else {
                for (int i = 0; i < 12;
i++) {
                    if (monthName[i] ==
monthStr)
                        currentMth =
monthNum[i];
                }
                currentYr = stoi(yearStr);
                //check for card validity
                if ((cardYr == currentYr &&
cardmth < currentMth) || cardYr < currentYr) {
                    setColor(4);
                    cout << "Invalid
card, expired!\n";
                    setColor(7);
                    continue;
                }
                else
                    break;
            }
        }
    }

    while (true) {
        cout << left << setw(25) << "Security
Code" << ":";
        getline(cin, cardSeccode);
    }
}

```

```

        if (!validateDigit(cardSeccode))
            continue;
        else if (cardSeccode.length() < 3 ||
cardSeccode.length() > 4) {
            setColor(4);
            cout << "Invalid Sevurity Code
Format! , Either 3 digits or 4 digits.\n";
            setColor(7);
            continue;
        }
        break;
    }
}
else {
    while (true) {
        cout << left << setw(25) << "Bank
Choice" << ":";

        getline(cin, bankChoice);

        if (!validateAlpha(bankChoice))
            continue;
        else
            break;
    }
    while (true) {
        cout << left << setw(25) << "Bank
Account Number" << ":";

        getline(cin, bankAccnum);

        if (!validateDigit(bankAccnum))
            continue;
        else if (bankAccnum.length() < 10 ||
bankAccnum.length() > 17) {
            setColor(4);
            cout << "Invalid Bank Account
Number Length. Must be between 10 - 17 digits\n";
            setColor(7);
            continue;
        }
        break;
    }
    while (true) {
        cout << left << setw(25) << "Bank
Account Pin" << ":";

        getline(cin, bankPin);

        if (!validateDigit(bankPin))
            continue;
        else if (bankPin.length() != 6) {
            setColor(4);
            cout << "Bank Account Pin is 6
digits.\n";

            setColor(7);

```



```

                                continue;
                                }
                                break;
                                }
                                }
                                cout << "-----
\n";

                                while (true) {
                                cout << "Confirm Transaction? (y=yes, n-
no) :";

                                getline(cin, pay);

                                if (!validateConfirmation(pay, confirm))
                                    continue;

                                else
                                    break;

                                }
                                cout << "-----
\n";

                                if (confirm == 'n') {
                                    cout << "Returning to menu....." << endl;
                                    loadingPlaneBar(200);
                                    return;
                                }
                                else {
                                    setColor(2);
                                    cout << "Transaction Successful!" << endl;
                                    setColor(7);

                                    currentUserstatus.paymentStatus = "Paid";
                                    ifstream inFile(currentUser +
"_paymentCheckin.txt");

                                    //read the content of file into an array
                                    while (getline(inFile, lines[count])) {
                                        //update the targeted data in the
array and record the number of data read in as count.
                                        lines[0] = to_string(ttlprice);
                                        if (lines[count] == oldText)
                                            lines[count] =
currentUserstatus.paymentStatus;

                                        count++;
                                    }
                                    inFile.close();

                                    //update the file by rewriting the updated
content from the array
                                    ofstream outFile(currentUser +
"_paymentCheckin.txt");

                                    for (int i = 0; i < count; i++)
                                        outFile << lines[i] << endl;

```

```

        outFile.close();

        system("pause");
    }
}

void checkIn(passengerStatus& currentUserstatus) {
    string proceed, firstName, lastName, passportNum,
    same, C_firstName, C_lastName, phoneNum, index, checkin;
    int idx;
    char copy, confirm;
    //no booking made = can't check in
    ifstream inFile(currentUser + "_Booking.txt");
    if (!inFile.is_open()) {
        setColor(4);
        cout << "No Booking Available, please book
your flight ticket(s)!\n";
        setColor(7);
        system("pause");
        return;
    }
    inFile.close();

    ///payment done = can't check in
    cout << "Payment Status: " <<
currentUserstatus.paymentStatus << endl;

    if (currentUserstatus.paymentStatus != "Paid") {
        setColor(4);
        cout << "Payment Must be Made Before Check
In!\n";
        setColor(7);
        system("pause");
        while (true) {
            cout << "Would You Like to Proceed to
Payment? (y=yes / n=no): ";

            getline(cin, proceed);
            if (!validateConfirmation(proceed,
confirm))

                continue;
            else
                break;
        }
        system("cls");
        if (confirm == 'n') {
            cout << "Returning to Main Menu.";
            loadingPlaneBar(200);
            return;
        }
        else {
            cout << "Going to Payment\n";
            system("pause");
            payment();
        }
    }
}

```

```

    }
    // checked in already = can't check in again
    else if (currentUserstatus.checkinStatus ==
"Checked In") {
        setColor(2);
        cout << "You Have Already Paid and Checked
in.\nThank You for Your Purchase." << endl;
        setColor(7);
        system("pause");
        system("cls");
        return;
    }

    else {
        cout << "~Proceed to Check In~\n";
        system("pause");
        for (int i = 0; i < noTickets; i++) {
            cout << "\nPassenger " << i + 1 <<
endl;

            //repeatedly prompt user to input
            passenger name until it is the same as booking file
            do {
                while (true) {
                    cout << left << setw(20) <<
"First Name" << ":";

                    getline(cin, firstName);

                    if
(!validateAlpha(firstName))
                        continue;
                    else {
                        currentUserstatus.passenger[i].passengerFullname.f
_Name = firstName;

                        break;
                    }
                }
                while (true) {
                    cout << left << setw(20) <<
"Last Name" << ":";

                    getline(cin, lastName);

                    if
(!validateAlpha(lastName))
                        continue;
                    else {
                        currentUserstatus.passenger[i].passengerFullname.l
_Name = lastName;

                        break;
                    }
                }
            }
        }
    }
}

```

```

    }
    } while (!checkSameName(firstName,
lastName, bookingList, i));

    cout << "Passport Number (eg.A12345)"
    << ":";

    while (true) {
        getline(cin, passportNum);

        if (!checkPassport(passportNum))
        {
            continue;
        }
        else
            break;
    }

    currentUserstatus.passenger[i].passportNum =
passportNum;

    //not first passenger
    if (i > 0) {
        while (true)
        {
            cout << "Same Contact
Person as Previous? (y=yes, n=no): ";

            getline(cin, same);
            if
            (!validateConfirmation(same, copy))
            {
                continue;
            }
            else
                break;
        }
        if (copy == 'y') {
            //second passenger can only
            //others can choose from 1
            if (i > 1) {
                while (true) {
                    cout <<
                    "\nWhich Same Contact Person?: ";
                    getline(cin,
                    index);

                    if
                    (!validateDigit(index) && index.length() != 1) {
                        cout <<
                        "\nEnter a Number Between (1-" << i << ")";
                        continue;
                    }
                    else if
                    (stoi(index) >= i) {
                        cout <<
                        "\nInvalid choice, Please Choose Between (1-" << i << ")";
                    }
                }
            }
        }
    }
}

```

```

                                                    continue;
                                                }
                                                else {
                                                    idx =
stoi(index);
                                                    break;
                                                }
                                            }
                                        }
                                        else
                                            idx = 0;
                                        C_firstName =
currentUserstatus.passenger[idx].contactPerson.f_Name;
                                        C_lastName =
currentUserstatus.passenger[idx].contactPerson.l_Name;
                                        phoneNum =
currentUserstatus.passenger[idx].contactPerson_Mbnum;

                                        currentUserstatus.passenger[i].contactPerson.f_Nam
e = C_firstName;

                                        currentUserstatus.passenger[i].contactPerson.l_Nam
e = C_lastName;

                                        currentUserstatus.passenger[i].contactPerson_Mbnum
= phoneNum;

                                        cout << "Contact Person
First Name: " << C_firstName;
                                        cout << "\nContact Person
Last Name: " << C_lastName;
                                        cout << "\nContact Person
Phone Number: " << phoneNum;
                                        cout << "\n-----
-----\n";
                                        continue;
                                    }
                                }

                                while (true) {
                                    cout << "Contact Person First
Name: ";
                                    getline(cin, C_firstName);

                                    if (!validateAlpha(C_firstName))
                                        continue;
                                    else {
                                        currentUserstatus.passenger[i].contactPerson.f_Nam
e = C_firstName;
                                        break;
                                    }
                                }

```

```

    }
}

while (true) {
    cout << "Contact Person Last
Name: ";

    getline(cin, C_lastName);

    if (!validateAlpha(C_lastName))
        continue;
    else {

        currentUserstatus.passenger[i].contactPerson.l_Nam
e = C_lastName;

        break;
    }
}

while (true) {
    cout << "Contact Person Phone
Number: ";

    getline(cin, phoneNum);

    if (!validateDigit(phoneNum))
        continue;
    else if (phoneNum.length() < 10
|| phoneNum.length() > 11) {
        cout << "Invalid Phone
Number Length!";

        continue;
    }
    else {

        currentUserstatus.passenger[i].contactPerson_Mbnum
= phoneNum;

        break;
    }
}

}

while (true) {
    cout << "Confirm Check in? (y=yes, n-
no): ";

    getline(cin, checkin);

    if (!validateConfirmation(checkin,
confirm))

        continue;
    else
        break;
}

if (confirm == 'n') {
    cout << "Returning to Main Menu.";

```

```

        loadingPlaneBar(200);
        return;
    }
    else {
        currentUserstatus.checkinStatus =
"Checked In";

        cout << "checked in";

        ofstream outFile(currentUser +
"_paymentCheckin.txt");

        //Update check in status of the
paymentCheckin text file for the current user
        if (outFile.is_open()) {
            outFile << fixed <<
setprecision(0) << currentUserstatus.ttlFlighttics << '\n';
            outFile <<
currentUserstatus.paymentStatus << '\n';
            for (int i = 0; i < noTickets;
i++) {
                outFile <<
currentUserstatus.passenger[i].passengerFullname.f_Name << '\n'
                <<
currentUserstatus.passenger[i].passengerFullname.l_Name << '\n'
                <<
currentUserstatus.passenger[i].passportNum << '\n'
                <<
currentUserstatus.passenger[i].contactPerson.f_Name << '\n'
                <<
currentUserstatus.passenger[i].contactPerson.l_Name << '\n'
                <<
currentUserstatus.passenger[i].contactPerson_Mbnum << '\n';
            }
            outFile <<
currentUserstatus.checkinStatus;
        }
        outFile.close();
    }
}

void printInvoice(bookingDetails flight[], passengerStatus passenger) {
    const int flights = 8;

    double flightPrice[flights] =
{ 200,200,200,200,250,250,300,300 };
    string Route[flights] = { "KL - Penang" , "Penang
- KL" , "KL - Johor" , "Johor - KL" ,
                                "KL -
Singapore" , "Singapore - KL" , "KL - Bangkok" , "Bangkok - KL" };
    string Time[4] = { "8:00 A.M.", "13:00 P.M.", "18:00
P.M.", "23:00 P.M." };

    ofstream outFile(currentUser + "_Invoice.txt");

```

```

        ifstream inFile(currentUser + "_Booking.txt");

        //Cant print invoice if no booking is made or
payment is not made yet
        if (!inFile.is_open()) {
            setColor(4);
            cout << "No Booking Available, please book
your flight ticket(s)!" << endl;
            setColor(7);
            system("pause");
            return;
        }
        else if (currentUserstatus.paymentStatus !=
"Paid") {

            setColor(4);
            cout << "You Have Not Paid yet!" << endl;
            setColor(7);
            system("pause");
            return;
        }
        inFile.close();

        //print the invoice
        outFile << "        Welcome to JSJK Airline Company"
<< endl;

        outFile <<
"=====INVOICE=====\\n";
        outFile << left << setw(20) << "Departure Flight"
<< setw(3) << ":" << Route[flight[0].depFlightnum - 1] << endl
            << setw(20) << "Departure Date" << setw(3)
<< ":" << flight[0].depDate << endl
            << setw(20) << "Departure Slot" << setw(3)
<< ":" << Time[flight[0].depTime - 1] << endl
            << setw(20) << "Return Flight" << setw(3) <<
":" << Route[flight[0].retFlightnum - 1] << endl
            << setw(20) << "Return Date" << setw(3) <<
":" << flight[0].retDate << endl
            << setw(20) << "Return Slot" << setw(3) <<
":" << Time[flight[0].retTime - 1] << endl;

        for (int i = 0; i < noTickets; i++) {
            outFile << "\\nPassenger " << i + 1 << endl;
            outFile << left << setw(25) << "Name" <<
setw(3) << ":" << currentUserstatus.passenger[i].passengerFullname.f_Name
+ " " + currentUserstatus.passenger[i].passengerFullname.l_Name << endl;
            outFile << setw(25) << "Passport Number" <<
setw(3) << ":" << currentUserstatus.passenger[i].passportNum << endl
            << setw(25) << "Contact Person" <<
setw(3) << ":" << currentUserstatus.passenger[i].contactPerson.f_Name + "
" + currentUserstatus.passenger[i].contactPerson.l_Name << endl <<
            setw(25) << "Contact Person Mobile No"
<< setw(3) << ":" << currentUserstatus.passenger[i].contactPerson_Mbnum <<
endl;

```



```

    }
    outFile.close();
    setColor(4);
    cout << "Invoice Printed, Please Check Your
Folder!" << endl;
    setColor(7);
    system("pause");
}

//Function to validate a name
bool isValidName(const string& name) {

    //Check if the name is empty
    if (name.empty())
        return false;

    int alphaCount = 0;

    for (int i = 0; i < (int)name.length(); ++i) {
        char ch = name[i];

        if (isalpha(ch)) {
            ++alphaCount;
        }

        //check if the name is invalid except for
        space, hypen and apostrophe
        else if (ch != ' ' && ch != '-' && ch !=
'\''') {
            return false;
        }
    }

    //has at least one alphabetic to be valid
    return alphaCount > 0;
}

//Function to validate a mobile number
bool isValidMobile(const string& mobile) {
    //check if mobile number is empty
    if (mobile.empty())
        return false;

    //check if the mobile num is exactly 10 digits
    if (mobile.length() < 10)
        return false;

    //check each character to ensure all the input is
    a digit
    for (int i = 0; i < (int)mobile.length(); ++i) {
        if (!isdigit(mobile[i]))
            return false;
    }
}

```

```

        return true;
    }

    //Function to validate an email
    bool isValidEmail(const string& email) {
        //to store the position of '@'
        int atIndex = -1;
        // to store the position of '.' after '@'
        int dotIndex = -1;

        //to find the first '@'
        for (int i = 0; i < email.length(); ++i) {
            if (email[i] == '@') {
                atIndex = i;
                break;
            }
        }

        //check if '@' is not the first or last character
        if (atIndex == -1 || atIndex == 0 || atIndex ==
email.length() - 1)
            return false;

        //to find the first '.'after the character '@'
        for (int i = atIndex + 1; i < email.length(); ++i)
        {
            if (email[i] == '.') {
                dotIndex = i;
                break;
            }
        }

        //check if '.' not before ,right after '@' or at
the end
        if (dotIndex == -1 || dotIndex == atIndex + 1 ||
dotIndex == email.length() - 1)
            return false;

        return true;
    }

    //Function to validate a username
    bool isValidUsername(const string& username) {
        //check if username is not empty, not less than 3
or more than 20 character
        if (username.empty() || username.length() < 3 ||
username.length() > 20)
            return false;

        //to ensure the first character is an alphabetic
        if (!isalpha(username[0]))
            return false;

        for (int i = 0; i < username.length(); ++i) {

```

```

        char u_name = username[i];

        //check if username is not an alphabetic or
digit        if ((!isalpha(u_name) && !isdigit(u_name)))
            return false;
        }

        return true;
    }

//Function to validate a password
bool isValidPassword(const string& password) {
    if (password.length() < 8)
        return false;

    int upper = 0, digit = 0, symbol = 0;

    for (int i = 0; i < password.length(); ++i) {
        char pw = password[i];
        upper += isupper(pw);
        digit += isdigit(pw);
        symbol += ispunct(pw);
    }

    return upper > 0 && digit > 0 && symbol > 0;
}

//Function to check the input is alphabet or not
bool checkAlphabet(string inputString) {
    for (int i = 0; i < inputString.length(); i++) {
        if (!isspace(inputString[i]) &&
        isdigit(inputString[i])) {
            setColor(4);
            cout << "Invalid input.Please enter
alphabets" << endl;

            cout << endl;
            setColor(7);
            return false;
        }
    }
    return true;
}

//Function to check the flight number is in the range 1 to 8 and is digit
only
bool checkFlightNum(int inputInteger) {
    if (cin.fail() || inputInteger < 1 ||
inputInteger > 8) {
        cin.clear();
        cin.ignore(1000, '\n');
        setColor(4);
        cout << "Invalid input. Please enter number
between 1-8" << endl;
    }
}

```

```

        setColor(7);
        cout << endl;
        return false;
    }

    return true;
}

//Function to check the flight number is in the range 1 to 4 and is digit
only
bool checkSlotNum(int inputInteger) {
    if (cin.fail() || inputInteger < 1 ||
inputInteger > 4) {
        cin.clear();
        cin.ignore(1000, '\n');
        setColor(4);
        cout << "Invalid input. Please enter number
between 1-4" << endl;

        cout << endl;
        setColor(7);
        return false;
    }

    return true;
}

//Function to check the date is in correct format
bool checkDate(string inputString){
    int dayList[12] = { 31, 0, 31, 30, 31, 30, 31, 31,
30, 31, 30, 31 };

    if (inputString.length() != 10){ //must is the
correct format

        setColor(4);
        cout << "Invalid Input. Please enter the
date with format (DD/MM/YYYY). " << endl;
        cout << endl;
        setColor(7);
        return false;
    }

    if (inputString[2] != '/' || inputString[5] !=
'/'){ // the index two and five of the date must be '/'
        setColor(4);
        cout << "Invalid Input. Please enter the
date with format (DD/MM/YYYY). " << endl;
        cout << endl;
        setColor(7);
        return false;
    }

    for (int i = 0; i < inputString.length(); i++) {
        if (i == 2 || i == 5)
            continue;

```

```

        if (!isdigit(inputString[i])){ //check is
digit or not
            setColor(4);
            cout << "Invalid Input. Please enter
only numbers in DD/MM/YYYY format." << endl;
            cout << endl;
            setColor(7);
            return false;
        }
    }

    //change the string into integer and separate the
day, month, year
    int day = stoi(inputString.substr(0, 2));
    int month = stoi(inputString.substr(3, 2));
    int year = stoi(inputString.substr(6, 4));

    //month must between 1-12
    if (month < 1 || month > 12) {
        setColor(4);
        cout << "Invalid Input. Please enter the
month between 1-12. " << endl;
        cout << endl;
        setColor(7);
        return false;
    }

    //year must between the normal range, not too
early, not too far
    else if (year < 2020 || year > 2030) {
        setColor(4);
        cout << "Invalid Input. Please enter the
year in the valid range. " << endl;
        cout << endl;
        setColor(7);
        return false;
    }

    //check the year whether is leap year or not
    else if (month == 02) {
        if (year % 4 == 0 && year % 100 != 0 ||
year % 400 == 0)
            dayList[1] = 29;

        else
            dayList[1] = 28;
    }

    //day must between correct range
    if (day > dayList[month - 1]) {
        setColor(4);
        cout << "Invalid Input. Please enter the day
is the valid range. " << endl;

```

```

        cout << endl;
        setColor(7);
        return false;
    }
    return true;
}

// Function to check the date of return flight is late than date of
departure flight
bool checkValid(string inputString, string inputfordate) {

    //change all the string into integer and separate
the day, month, year
    int dayDeparture = stoi(inputfordate.substr(0,
2));
    int monthDeparture = stoi(inputfordate.substr(3,
2));
    int yearDeparture = stoi(inputfordate.substr(6,
4));

    int dayReturn = stoi(inputString.substr(0, 2));
    int monthReturn = stoi(inputString.substr(3, 2));
    int yearReturn = stoi(inputString.substr(6, 4));

    //check the year
    if (yearReturn < yearDeparture) {
        setColor(4);
        cout << "Invalid input. Return date must be
the same as or after the departure date. " << endl;
        cout << endl;
        setColor(7);
        // if year of date of return flight is early
than date of departure flight, return false
        return false;
    }

    else if (yearReturn == yearDeparture) {
        //check month
        if (monthReturn < monthDeparture) {
            setColor(4);
            cout << "Invalid input. Return date
must be the same as or after the departure date. " << endl;
            cout << endl;
            setColor(7);
            // if month of date of return flight
is early than date of departure flight, return false
            return false;
        }

        //check day
        else if (monthReturn == monthDeparture) {
            if (dayReturn < dayDeparture) {
                setColor(4);
                cout << "Invalid input. Return
date must be the same as or after the departure date. " << endl;
            }
        }
    }
}

```

```

        cout << endl;
        setColor(7);
        // if day of date of return
flight is early than date of departure flight, return false
        return false;
    }
}
}
return true;
}

// Function to check the input whether is same with the first passenger
(integer only)
bool checkSamedetailInt(int inputInteger, int firstinputInt, int count) {
    if (count == 0)
        return true; //if the passenger is the
first, return true
    else
        if (inputInteger != firstinputInt) {
            setColor(4);
            cout << "The input must same with the
previous passenger. " << endl;
            cout << endl;
            setColor(7);
            return false;
        }
        return true;
}

// Funtion to check the input whether is same with the first passenger
(string only)
bool checkSamedetailStr(string inputString, string firstinputStr, int
count) {
    if (count == 0)
        return true; //if the passenger is the
first, return true
    else
        if (inputString != firstinputStr) {
            setColor(4);
            cout << "The input must same with the
previous passenger. " << endl;
            cout << endl;
            setColor(7);
            return false;
        }
        return true;
}

// Function to check the input whether is same with the original
information (integer only)
bool checkNochangeInt(int originalInteger, int inputInteger) {
    if (inputInteger == originalInteger) {
        setColor(4);

```

```

        cout << "Oops! The new number you entered is
the same as the previous one." << endl;
        cout << "Please enter a different number."
<< endl;

        cout << endl;
        setColor(7);

        //if the information is same, return false
        return false;
    }
    return true;
}

// Function to check the input whether is same with the original
information (string only)
bool checkNochangeDate(string originalString, string inputString) {
    if (inputString == originalString) {
        setColor(4);
        cout << "Oops! The new date you entered is
the same as the previous one." << endl;
        cout << "Please enter a different date." <<
endl;

        cout << endl;
        setColor(7);

        //if the information is same, return false
        return false;
    }
    return true;
}

// Function to check the name is different from original name which they
would like to change and other passengers' name
bool checkallName(string inputfirstName, string inputlastName,
bookingDetails bookingList[], int currentIndex) {
    string fullName = inputlastName + inputfirstName;

    for (int count = 0; count < noTickets; count++) {
        string existingFullName =
bookingList[count].passengerFullname.l_Name +
bookingList[count].passengerFullname.f_Name;

        if (count == currentIndex) {
            // Check if name is same as the
original
            if (fullName == existingFullName) {
                setColor(4);
                cout << "The name is the same as
the previous one." << endl;

                cout << "Please enter another
name." << endl << endl;

                setColor(7);
                return false;
            }

```



```

        }
        else {
            // Check if the name is used by
another passenger
            if (fullName == existingFullName) {
                setColor(4);
                cout << "This name is already
used in another booking." << endl;
                cout << "Please enter another
name." << endl << endl;
                setColor(7);
                return false;
            }
        }
    }
    return true;
}

//Function to check the name is different from other passengers
bool checkSameName(string inputfirstName, string inputlastName,
bookingDetails bookingList[], int currentIndex) {
    string fullName = inputlastName + inputfirstName;

    for (int count = 0; count < currentIndex; count++)
    {
        string existingFullName =
bookingList[count].passengerFullname.l_Name +
bookingList[count].passengerFullname.f_Name;

        if (fullName == existingFullName) {
            setColor(4);
            cout << "Sorry. This full name already
exists in another booking." << endl;
            cout << "Please enter another name."
<< endl << endl;
            setColor(7);
            return false;
        }
    }
    return true;
}

//Function to check the input is between number 1-6
bool checkselectNum(int select) {
    //Ensure the input is not alphabet and in the
range
    if (cin.fail() || select < 1 || select > 6) {
        cin.clear();
        cin.ignore(1000, '\n');
        setColor(4);
        cout << "Please enter 1-6 only!!" << endl;
        cout << endl;
        system("pause");
        system("cls");
    }
}

```

```

        setColor(7);
        return false;
    }
    return true;
}

bool validateAlpha(string& input) {
    string trimmed;
    int start, end;
    bool inAword = 0, cap = true;
    //check for empty input then input type
    if (!input.empty()) {
        for (int i = 0; i < input.length(); i++) {
            if (!isalpha(input[i])
&& !isspace(input[i])) {
                setColor(4);
                cout << "Invalid input, please
enter again\n";

                input.clear();
                setColor(7);
                return 0;
            }
        }
        //start to clean up the string input
        //find the index of the first and last non-
space character

        start = input.find_first_not_of(" ");
        end = input.find_last_not_of(" ");
        //trim the input from start and end
        input = input.substr(start, end - start +
1);

        //clean the spaces in the middle
        for (int i = 0; i < input.length(); i++) {
            if (isspace(input[i])) //the current
character is a space

            {
                if (inAword) //were in a word,
so add the space

                {
                    trimmed += " ";
                    inAword = 0; // update as
were not in a word

                }
                else
                    continue;
            }
            else//add non space character to the
trimmed string

            {
                trimmed += input[i];
                inAword = 1; //are/were in a
word

            }
        }
    }
}

```

```

    }
    //update parameter input
    input = trimmed;

    //capitalise each word in the input
    for (int j = 0; j < input.length(); j++) {
        if (isspace(input[j])) {
            cap = true;
        }
        else if (cap) {
            input[j] = toupper(input[j]);
            cap = false;
        }
        else {
            input[j] = tolower(input[j]);
        }
    }
    return 1;
}
else //is empty
    return 0;
}

bool validateConfirmation(string input, char& choice) {
    //check for empty input, invalid input type and
    alphabets.
    if (input.empty() || !validateAlpha(input) ||
    input.length() != 1 || (tolower(input[0]) != 'y' && tolower(input[0]) !=
    'n')) {
        setColor(4);
        cout << "Please follow the instructions.\n";
        //reset the string if its invalid.
        input.clear();
        setColor(7);
        return 0;
    }
    else {
        choice = tolower(input[0]);
        return 1;
    }
}

bool validateDigit(string input) {
    //check for empty input then input type
    if (!input.empty()) {
        for (int i = 0; i < input.length(); i++) {
            if (!isdigit(input[i])) {
                setColor(4);
                cout << "Invalid input, please
enter the correct numbers only!\n";
                setColor(7);
                input.clear();
                return 0;
            }
        }
    }
}

```

```

        }
        return 1;
    }
    else
        return 0;
}

bool checkPassengername(string Fname, string Lname, bookingDetails
bookingList[], int idx) {
    string inputPass = Fname + Lname,
        correctPass =
bookingList[idx].passengerFullname.f_Name +
bookingList[idx].passengerFullname.l_Name;
    if (inputPass == correctPass)
        return 1;
    else {
        setColor(4);
        cout << "Incorrect Passenger Name, Please
Enter According to Your Booking.\n";
        setColor(7);
        return 0;
    }
}

bool checkPassport(string& passport) {
    string nums;
    bool hasNums;
    //check for input length
    if (passport.length() != 6) {
        setColor(4);
        cout << "Invalid Passport Format!\n";
        setColor(7);
        return 0;
    }
    //determine the number part of the input if it is
correct length
    nums = passport.substr(1, 5);

    //check if all of the number part are numbers
    for (int i = 0; i < 5; i++) {
        if (!isdigit(nums[i])) {
            setColor(4);
            cout << "Invalid Passport Format!\n";
            //wrong number part
            hasNums = 0;
            setColor(7);
            break;
        }
    }
    //correct number part
    hasNums = 1;
    //check for format

```

```

        if (!isalpha(passport[0]) || !hasNums) {
            setColor(4);
            cout << "Invalid Passport Format!\n";
            setColor(7);
            return 0;
        }
        else {
            passport[0] = toupper(passport[0]);
            return 1;
        }
    }

void showLoading() {
    cout << endl;
    setColor(6);
    cout << "Loading";
    for (int i = 0; i < 3; i++) {
        Sleep(1000); // wait 1 second
        cout << ".";
        cout.flush(); // make sure dot prints
        immediately
    }
    cout << endl;
    setColor(7);
}

void loadingPlaneBar(int delayMs) {
    string wing = "    _|_";
    string plane = "---o--(_)--o---";
    int barWidth = 50;
    for (int i = 0; i <= 15; ++i) {
        // Clear screen instead of using cursor
        moves
#ifdef _WIN32
            system("CLS");
        #else
            system("clear");
        #endif

        int planePos = (i * barWidth) / 15;

        cout << string(planePos, ' ') << wing <<
        "\n";
        cout << string(planePos, ' ') << plane <<
        "\n";

        cout << "    [";
        int progress = (i * barWidth) / 15;

        setColor(2); // Optional color
        cout << string(progress, '#');
        cout << string(barWidth - progress, ' ');
        setColor(7); // Reset color
        cout << "]" << "\n";
    }
}

```